

Agentic Tool Use in Large Language Models: A Survey

HU JINCHAO, School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China

MEIZHI ZHONG, Independent Researcher, China

KEHAI CHEN(CORRESPONDING AUTHOR), School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China

XUEFENG BAI, School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China

MIN ZHANG, School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China

Large language models are increasingly being deployed as autonomous agents yet their real world effectiveness depends on reliable tools for information retrieval, computation and external action. Existing studies remain fragmented across tasks, tool types, and training settings, lacking a unified view of how tool-use methods differ and evolve. This survey organizes the literature into three paradigms: prompting as plug-and-play, supervised tool learning and reward-driven tool policy learning, analyzes their methods, strengths and failure modes, reviews the evaluation landscape and highlights key challenges, aiming to address this fragmentation and provide a more structured evolutionary view of agentic tool use.

CCS Concepts: • **Computing methodologies** → **Natural language generation**.

Additional Key Words and Phrases: Large language models, Agentic tool use, Prompt engineering, Instruction tuning, Reinforcement learning

1 Introduction

Artificial intelligence (AI) has evolved from early rule-based expert systems to modern data-driven machine learning [12]. The Transformer architecture [139] catalyzed this transition and enabled the rise of large language models (LLMs). Foundational models such as BERT [31] and T5 [115] established strong paradigms for natural language understanding and generation. Subsequent models, including Llama 2 [137], Qwen [114], Mixtral [55], and GPT-4 [97], have demonstrated unprecedented capability. Pretrained on massive corpora [36], these models exhibit emergent strengths [152] in contextual understanding, complex reasoning [66], and content generation.

However, standard LLMs remain inherently static. Their knowledge is bounded by training data, which can lead to factual errors or "hallucinations" [70]. They also struggle to access real-time information, execute precise computation, or interact reliably with external systems [54]. These limitations, together with safety and bias risks inherited from web-scale data [10], constrain deployment in dynamic real-world settings.

To overcome these boundaries, agentic AI has emerged as a central paradigm [89, 142, 193]. In this paradigm, the LLM serves as a reasoning core but is augmented with planning, memory, and action capabilities. Tool use is the key mechanism that operationalizes action [121, 122]. By invoking external tools (e.g., APIs, databases, and code interpreters), an LLM agent can access up-to-date knowledge through search [92], improve computational accuracy through programmatic execution [37], and perform real-world operations through external systems [122].

Authors' Contact Information: Hu Jinchao, School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, Shenzhen, China, jchu@stu.hit.edu.cn; Meizhi Zhong, Independent Researcher, Beijing, China, meizhi.zhong.1999@gmail.com; Kehai Chen(corresponding author), School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, Shenzhen, China, chenkehai@hit.edu.cn; Xuefeng Bai, School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, Shenzhen, China, baixuefeng@hit.edu.cn; Min Zhang, School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, Shenzhen, China, zhangmin2021@hit.edu.cn.

Table 1. Comparison of representative survey papers on agentic tool use and function calling.

Survey Paper	Chronological evolution	Paradigm taxonomy	Tool coverage	Training data	Evaluation landscape	Open challenges
Zhao et al. [193]	✓	✗	✗	✗	✓	✓
Wang et al. [142]	✓	✗	✗	✗	✓	✓
Mialon et al. [89]	✓	✗	✗	✗	✓	✓
Qu et al. [113]	✗	✓	✓	✓	✓	✓
Shen [123]	✗	✓	✓	✓	✗	✓
Chen et al. [20]	✗	✓	✓	✓	✓	✓
Ours	✓	✓	✓	✓	✓	✓

The growing importance of tool use has driven substantial research activity, including multiple surveys that attempt to structure the field. Existing reviews cover complementary perspectives: some emphasize agent architecture (planning, memory, and action) [193]; others review augmented language models and external memory/tool mechanisms [89, 190]; and still others focus on tool taxonomies and data construction pipelines [113]. Despite their value, many of these frameworks provide a largely static snapshot. What remains insufficiently articulated is the methodological evolution from prompting as plug-and-play to supervised tool learning and reward-driven policy learning. This missing evolutionary view can obscure both continuity and trade-offs across paradigms. Table 1 compares representative surveys along six dimensions: chronological evolution, paradigm taxonomy, tool coverage, training data, evaluation landscape, and open challenges.

In this paper, we make several key contributions to the field of LLM-based agentic tool use:

- We provide an evolutionary synthesis of agentic tool use, tracing how the field progresses from prompting-based control to supervised tool learning and reward-driven policy learning.
- We propose a unified three-paradigm taxonomy with explicit methodological boundaries, clarifying the role of optimization signals (in-context control, supervised signals, and reward feedback).
- We systematize the evaluation landscape as a cross-cutting dimension, connecting method families to benchmark levels from function-call correctness to end-to-end interactive success, safety, and robustness.
- We identify unresolved challenges and practical research directions, including long-horizon credit assignment, scalable tool generalization, and alignment-aware deployment.

In this survey, we synthesize agentic tool use from an explicitly evolutionary perspective. We organize prior work into three methodological paradigms: (I) Prompting as Plug-and-Play, where a frozen model is guided to use tools through prompts and in-context interaction instead of weight updates [162]; (II) Supervised Tool Learning, which uses labeled or synthetic supervision to internalize robust tool-use behavior [122]; and (III) Reward-Driven Tool Policy Learning, which optimizes long-horizon interaction through reinforcement and reward feedback [125]. We further analyze evaluation as a cross-cutting dimension and track its evolution from function-call correctness [73] to end-to-end task success in interactive settings [80]. Through this lens, we aim to provide a coherent map of the field, clarify the design trade-offs linking paradigms, and connect foundational methods to emerging autonomous agent systems. Figure 1 summarizes this framework.

- **Prompting as Plug-and-Play.** This initial approach uses frozen LLM parameters and relies on in-context learning and prompt engineering. It requires no model training. This category explores control flows such as

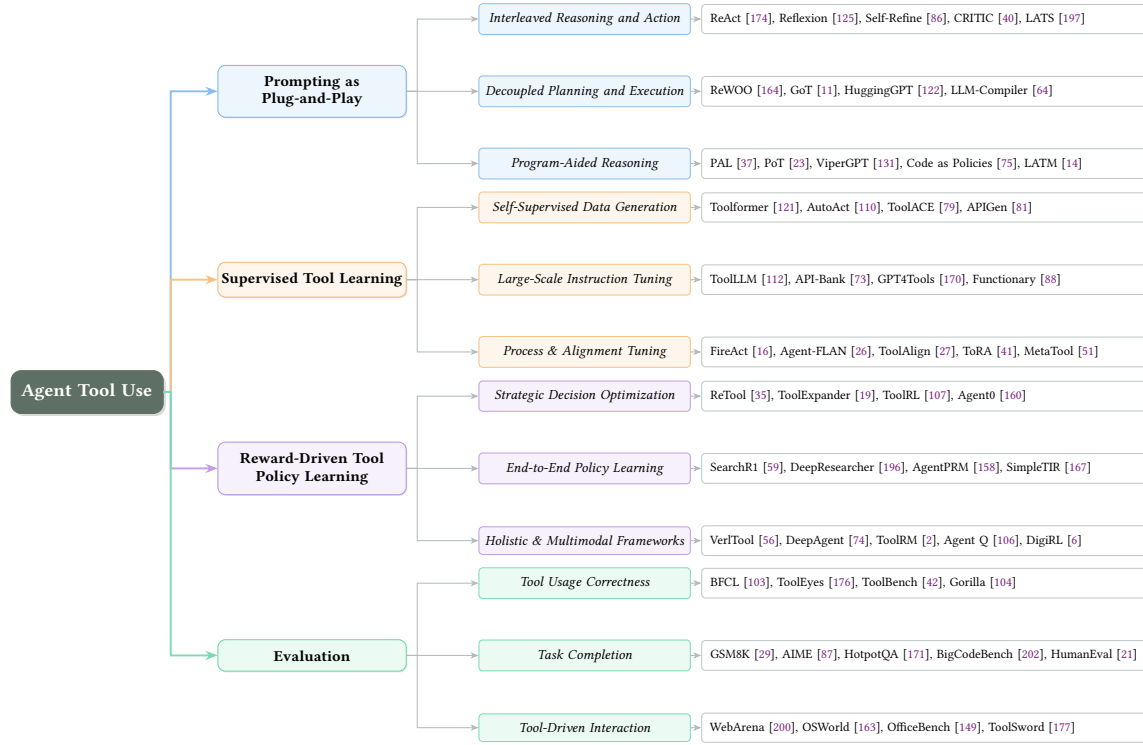


Fig. 1. Overview of the Agentic Tool Use framework.

interleaved reasoning and acting cycles that mix thought and action, and decoupled planning and execution that separates planning from action. While highly flexible, this method often suffers from high latency and token costs, and its reliability can be brittle for complex, multi step tasks.

- **Supervised Tool Learning.** This approach moves from prompt engineering to data engineering to overcome the scaling and reliability bottlenecks of tuning-free methods. The core idea is to use supervised fine-tuning (SFT) to teach models the syntax and logic of tool use, internalizing this skill into model parameters. The primary challenge is creating large-scale, high-quality datasets. Key strategies include self-supervised data generation and teacher-guided instruction synthesis for complex multi-step tool-use trajectories.
- **Reward-Driven Tool Policy Learning.** This paradigm shifts from static imitation to dynamic optimization, treating tool use as a sequential decision-making problem. It uses reinforcement learning (RL) to train the agent to discover an effective policy in dynamic environments rather than mimicking fixed trajectories. The research targets strategic decision making, including when to call a tool versus relying on model knowledge, and multi-tool credit assignment in long tool-call chains.
- **Evaluation.** This category summarizes evaluation for tool-using agents and has co-evolved with the methodological paradigms. Benchmarks progressed from function-call correctness (valid syntax and arguments) to end-to-end task success in interactive environments. Recent benchmarks further evaluate whether agents follow safety rules and operational guidelines while completing tasks.

Notably, this survey emphasizes the complementary nature of the three paradigms and argues that production-grade systems will likely combine all of them. We further summarize unresolved challenges and promising research directions, including multi-tool credit assignment in long-horizon tasks, autonomous tool creation and self-extension, and the integration of safety, alignment, and compliance into deployed agent systems. The organization of the rest of this survey is as follows. We begin with the background of this survey in [section 2](#), introducing the development timeline of LLM tool use. We then discuss three methodological paradigms: prompting as plug-and-play ([section 3](#)), supervised tool learning ([section 4](#)), and reward-driven tool policy learning ([section 5](#)). We treat evaluation as a cross-cutting dimension and summarize the science and benchmarks of evaluation in [section 6](#). In the end, we conclude this paper with a summary of the three complementary paradigms and a discussion about our future outlooks in [section 7](#).

2 Background of Agent Tool Use

In this section, we delineate the developmental trajectory of agentic tool use, tracing the evolution from rudimentary neuro-symbolic concepts to the sophisticated, self-optimizing agents of today. This progression encompasses the initial theoretical blueprints, the emergence of prompt-based reasoning frameworks, and the shift towards data-driven internalization and reinforcement learning. The key milestones that define this timeline are illustrated in Figure 2.

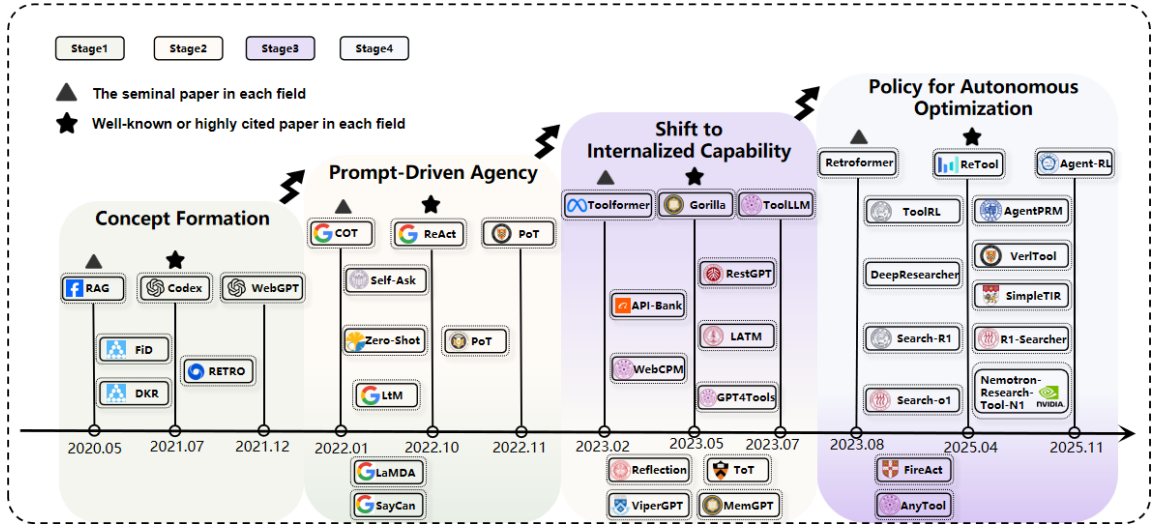


Fig. 2. Timeline of Agentic Tool Use.

2.1 Early Exploration and Concept Formation

The conceptual foundations of agentic tool use emerged before the rise of modern instruction-tuned LLMs. As researchers recognized that purely parametric models were limited in factual updating, precise computation, and interaction with external environments, early work began to augment language models with external resources rather than relying on scaling alone. One important step was Retrieval-Augmented Generation (RAG) [70], which showed that coupling a generator with non-parametric memory could improve knowledge access and reduce hallucinations. This line of thought was further generalized by MRKL [63], which framed the language model as a central router that could dispatch subproblems to specialized modules and symbolic systems.

At the same time, several works moved beyond static augmentation toward explicit interaction. WebGPT [92] demonstrated that a model could browse the web to gather evidence for long-form question answering, while LaMDA [135] incorporated tools such as a calculator and a translator into dialogue generation, showing the value of grounding responses in external utilities. Beyond information access, TALM [101] explored how smaller models could be trained to generate tool calls and use returned results through supervised learning, providing an early instance of API-oriented tool use.

The notion of tools also expanded beyond conventional APIs. Codex [21] suggested that language models could solve problems by generating executable code, effectively treating program execution as an external computational interface. In parallel, SayCan [4] extended this paradigm to embodied settings by combining language-model priors with robotic affordances, allowing LLMs to guide physical action selection. These early studies did not yet constitute general-purpose agents, but they established the key ingredients of later systems: external knowledge access, modular routing, executable actions, and grounded interaction with digital or physical environments.

2.2 The Emergence of Prompt-Driven Agency (2022-2023)

Building on these early explorations, researchers began to observe that large language models could exhibit effective tool-use behavior without modifying their parameters. This stage was characterized by prompt engineering and in-context learning, which made it possible to elicit agent-like behavior from frozen models through suitable reasoning and interaction scaffolds. Chain-of-Thought prompting [153] laid the foundation by showing that LLMs could solve complex tasks through explicit intermediate reasoning. Building on this idea, ReAct [174] connected reasoning with external action through a Thought-Action-Observation loop, allowing models to decide when to call tools and update subsequent steps based on returned observations. Self-Ask [105] further highlighted the value of decomposing complex questions into simpler subproblems, especially in tool-assisted question answering. As tasks became more challenging, Reflexion [125] introduced self-reflective verbal feedback to help agents learn from past failures, while Tree of Thoughts [173] extended linear reasoning into search over multiple candidate trajectories, improving planning flexibility.

In parallel, another line of work improved reliability by shifting parts of reasoning to executable intermediates. Program-Aided Language Models [37] and Program of Thoughts [23] used code generation to externalize computation, improving precision on arithmetic and symbolic problems. This idea was extended to multimodal and neural-symbolic settings by ViperGPT [131] and Binder [28], where code served as a structured interface for coordinating external tools. As tool ecosystems grew, researchers also explored more modular forms of orchestration. ART [100] selected from tool-augmented reasoning templates, Chameleon [82] treated tools as composable modules, ReWOO [164] decoupled planning from execution to reduce interaction overhead, and HuggingGPT [122] framed the LLM as a central controller over diverse expert models.

Prompt-driven agency also began to move beyond single-turn task solving toward persistent and open-ended behavior. Generative Agents [102] demonstrated the role of memory in sustained agent behavior, MemGPT [98] introduced a memory management mechanism for handling long-context interaction, and Voyager [141] showed that prompt-based agents could continuously acquire and refine skills in embodied environments. These works marked a distinct phase in which tool use, planning, and memory could be elicited primarily through prompting, establishing the practical foundation for later training-based and policy-driven agent systems.

2.3 The Shift to Supervised Tool Learning (2023-2024)

As prompt-based tool use became increasingly capable, its limitations also became more apparent. Repeated prompting introduced substantial latency and token overhead, while long reasoning-action trajectories often remained brittle in complex tasks. These challenges motivated a transition from eliciting tool use in context to internalizing it through training, allowing models to invoke tools more efficiently and reliably. Early efforts explored supervised imitation of tool-augmented behavior. WebCPM [111] showed that language models could be fine-tuned on human web-browsing trajectories for goal-directed interaction, and Toolformer [121] further demonstrated that models could teach themselves when and how to call APIs by inserting tool-use annotations into their own training data. This direction was extended to instruction-tuned settings by ToolAlpaca [133], which used synthetic supervision from stronger models, and by ToolLLM [112], which scaled training to thousands of real-world APIs.

As the number and diversity of tools increased, internalized tool use also required better mechanisms for selection, adaptation, and evaluation. API-Bank [73] provided an early benchmark for planning, tool calling, and response generation in API-based interaction. To improve scalability under large tool libraries, AnyTool [34] introduced hierarchical retrieval to narrow the candidate space before invocation. The dynamic nature of external interfaces posed another challenge, since APIs evolve over time and new tools appear continuously. Gorilla [104] addressed this issue by combining retrieval with API-aware fine-tuning, enabling models to use up-to-date documentation at inference time. Related efforts such as RestGPT [126] and GPT4Tools [170] further explored structured interaction with RESTful APIs and the generation of specialized tool-use data.

Later work increasingly focused on robustness, transfer, and the balance between agentic behavior and general capability. FireAct [16] showed that fine-tuning on interleaved reasoning-action trajectories could substantially improve the performance of smaller open models. Generalization to unseen tools was further studied by NexusRaven [127], which emphasized accurate function calling from tool descriptions, while APIGen [81] improved data quality through verifiable synthetic API-call generation. At the same time, researchers began to examine how tool specialization could be integrated with broader model competence. Lemur [166] proposed a unified framework that balanced function calling, tool planning, and general language understanding, and AgentTuning [183] further highlighted the importance of preserving generalist reasoning ability while optimizing for agentic behavior. These studies marked the transition from learning tool use through prompts to learning it in model weights, establishing supervised fine-tuning and synthetic data generation as practical routes toward more efficient, scalable, and robust tool-using agents.

2.4 The Rise of Policy for Autonomous Optimization (2024-Present)

Despite the effectiveness of supervised fine-tuning, an important limitation remained: static training signals were often insufficient for exploration, error recovery, and long-horizon adaptation in dynamic environments. As a result, recent research has increasingly turned to reinforcement learning, treating multi-turn tool use as a sequential decision-making problem optimized through environmental feedback rather than imitation alone. This stage can be understood as a shift from learning how to call tools to learning how to optimize interaction with them.

Early efforts in this direction focused on improving the quality of tool-use decisions under feedback. ReTool [35] applied reinforcement learning to help agents distinguish between cases that required external tools and those that could be solved from internal knowledge, thereby reducing unnecessary tool calls. Another important challenge was how to learn from failed executions. Retroformer [175] addressed this issue through retrospective learning over past tool-use trajectories, enabling agents to revise policies based on environmental outcomes. Along a similar line, TRICE

[109] tackled credit assignment in long tool chains by strengthening steps that contributed to correct final answers while suppressing noisy or unhelpful actions.

More recent work has moved toward jointly optimizing reasoning and tool-use policies at scale. Search-R1 [59] showed that reinforcement learning over the interaction between reasoning and retrieval could induce more effective search behavior than imitation-based methods alone. SimpleTIR [167] further unified reasoning and tool execution within a single end-to-end training loop, reducing the mismatch often introduced by pipeline-style designs. At the same time, RLFactory [15] and VerlTool [56] helped systematize this emerging paradigm by providing infrastructure for trajectory-level reward modeling and broader agentic reinforcement learning workflows. These studies suggest an emerging stage in which tool-using agents are optimized not only to invoke external resources, but also to refine their interaction strategies through feedback and trial-and-error.

3 Prompting as plug-and-play

Prompting as plug-and-play refers to tool-use methods that rely on prompting and in-context interaction rather than parameter updates. In this paradigm, the language model remains frozen and serves as the central reasoning and coordination module, while tool-use behavior is elicited through instructions, demonstrations, and feedback from external environments. This design offers several practical advantages, including flexibility, interpretability, and low deployment cost, making it particularly suitable for rapid prototyping and adaptation to newly introduced tools. At the same time, because the model does not internalize tool-use behavior in its parameters, these methods often suffer from higher latency and less stable performance in complex or long-horizon tasks. To better organize this paradigm, we group existing methods into three representative structures according to how reasoning, planning, and execution are coordinated: Interleaved Reasoning and Action, Decoupled Planning and Execution, and Program-Aided Reasoning. As illustrated in Figure 3, these three categories capture the main control flows through which frozen LLMs interact with external tools.

3.1 Interleaved Reasoning and Action

A representative structure within Paradigm I is interleaved reasoning and action, which treats problem solving as a multi-turn interaction between the agent and its environment. Rather than completing all reasoning before acting, the model alternates between intermediate deliberation and external operations, using observations from the environment to guide subsequent decisions. A representative starting point in this line is ReAct [174], which interleaves reasoning traces with task-specific actions so that the model can explicitly decide what to do next based on newly acquired information. This design helps mitigate the disconnect between abstract reasoning and external execution, and has become a canonical control loop for prompting-based tool use.

Subsequent work extended this interleaved loop in several directions. One line focused on improving robustness through self-correction and iterative refinement. Reflexion [125] augments the standard interaction loop with verbal self-reflection, allowing the agent to critique failed trajectories and reuse these reflections in later attempts, while Self-Refine [86] shows that a model can iteratively improve its own intermediate reasoning or final outputs through self-generated feedback. Another line introduced explicit verification mechanisms. Chain-of-Verification [32] prompts the model to generate verification questions for its own claims before producing a revised answer, and CRITIC [40] further grounds this process in external tools by validating intermediate or final outputs with search engines and other utilities. To better handle tasks that require foresight and exploration, LATS [197] combines the interaction loop of ReAct with tree search, enabling more deliberate exploration of alternative reasoning-action trajectories.

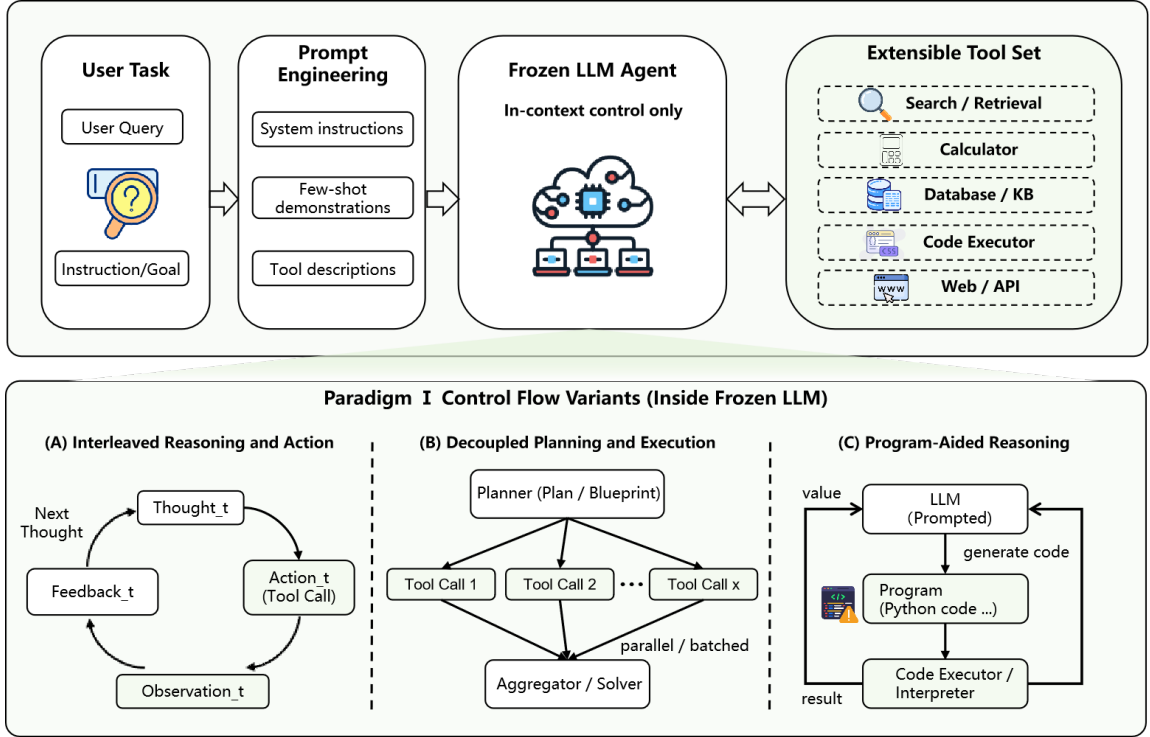


Fig. 3. Illustration of the Prompting as plug-and-play paradigm.

Related prompting techniques can further strengthen this paradigm, although they are not themselves interleaved tool-use frameworks. Self-Consistency [148] improves robustness by aggregating multiple sampled reasoning paths, Least-to-Most Prompting [199] encourages decomposition into simpler subproblems, and Take a Step Back [194] promotes abstraction before low-level action selection. These methods show how prompting alone can support a tightly coupled loop of reasoning, acting, observing, and revising, making interleaved control one of the most influential architectures in prompting-based agentic tool use.

3.2 Decoupled Planning and Execution

A second representative structure within Paradigm I is decoupled planning and execution, which separates high-level reasoning from downstream tool invocation. Compared with interleaved reasoning-action loops, this design reduces repeated deliberation at every step and can improve efficiency when tasks require long tool chains or multiple dependent operations. Instead of alternating between thought and action after each observation, the model first produces a structured plan and then executes it through external tools, allowing planning and execution to be handled more explicitly.

A representative starting point in this line is ReWOO [164], which decouples reasoning from observation by introducing a planner-worker architecture. The planner first generates a sequence of executable subtasks with variable assignments, and the worker then carries them out without requiring the model to replan after every intermediate observation. This design reduces interaction overhead and makes tool use more efficient. Subsequent work further

extended this idea toward richer scheduling and more adaptive control. LLMCompiler [64] organizes tool calls as a dependency-aware computation graph and enables parallel execution when subtasks are independent, while AdaPlanner [128] shows that decoupled planning can still remain adaptive by revising plans in response to environmental feedback.

Related methods explored neighboring forms of plan-first control. Plan-and-Solve Prompting [143] highlights the value of generating an explicit plan before solving a task, and Skeleton-of-Thought [94] similarly improves efficiency by producing a coarse outline before expanding details. In more structured settings, PEARL [129] applies a plan-and-execute strategy to long documents, allowing the model to decompose document-grounded tasks into manageable actions. Other systems extended decoupled control to broader tool orchestration. ART [100] retrieves suitable reasoning-action programs from an external task library, Chameleon [82] composes tools through modular execution pipelines, and systems such as HuggingGPT [122] and TaskMatrix.AI [76] further illustrate how an LLM can act as a central planner over diverse expert models and APIs. These methods show that separating planning from execution offers a practical alternative to tightly interleaved control, especially when tool use requires structured decomposition, reduced latency, or coordination across multiple modules.

3.3 Program-Aided Reasoning

The third sub-paradigm addresses the tendency of LLMs to make arithmetic and logical errors when reasoning purely in natural language by shifting the intermediate representation from text to executable code. In this setting, the external program executor is not merely an auxiliary utility, but becomes part of the reasoning process itself. PAL [37] and PoT [23] are representative starting points in this line, prompting the model to generate Python programs that externalize computation and decouple reasoning from execution. To further strengthen logical rigor, Logic-LM [99] translates natural language problems into symbolic formulations that can be solved by external logic solvers, improving determinism in deductive reasoning tasks.

This program-centric perspective was later extended to broader execution environments. Chain of Code [71] broadens code-based reasoning beyond purely executable computation by allowing the model to mix runnable code with flexible pseudo-code, so that semantic sub-tasks can still be handled through a language-model-augmented code emulator. ViperGPT [131] applies a similar philosophy to visual reasoning, where the model generates Python code to call vision APIs and compose multimodal operations. In embodied settings, Code as Policies [75] further shows that natural language instructions can be compiled into robot policy code through predefined control APIs, extending program-aided reasoning into action generation.

The same idea also motivates more advanced forms of tool creation and reuse. MathPrompter [53] improves reliability by generating multiple candidate Python functions and checking for agreement across their execution results. LATM [14] goes one step further by enabling agents to create reusable Python tools for unseen tasks, while CREATOR [108] further formalizes this process by separating tool creation from downstream decision execution. These methods show that prompting can use code not only as an execution backend, but also as a structured medium for reasoning, control, and even tool invention itself.

4 Supervised Tool Learning

Compared with prompting-only approaches, this paradigm shifts the focus from externally eliciting tool-use behavior at inference time to internalizing it through training. Rather than relying on prompts alone to scaffold reasoning and tool invocation, Supervised Tool Learning uses labeled or synthetic supervision to encode tool-use patterns directly

into model parameters. This transition improves efficiency, stability, and scalability, while also enabling models to generalize more reliably across diverse tools and task settings. In this setting, the central challenge is not only how to obtain training data at scale, but also how to design supervision signals that capture tool syntax, invocation semantics, planning structure, and alignment requirements. As illustrated in Figure 4, these categories together show how different forms of supervision enable models to acquire and internalize tool-use capability.

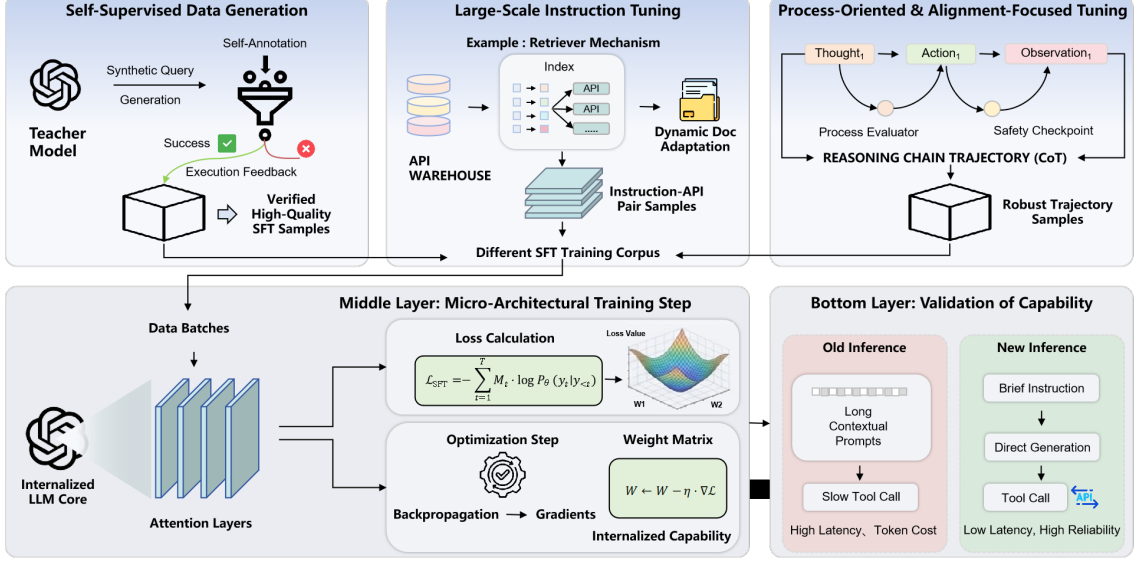


Fig. 4. Training Pipeline of Paradigm II: From Supervision to Internalized Tool-Use Capability.

4.1 Self-Supervised Data Generation

A natural starting point for Paradigm II is to reduce the dependence on costly human annotation by automatically constructing tool-use supervision. Early efforts explored self-supervised data generation, where models learn tool-use patterns by labeling or simulating their own interaction traces rather than relying entirely on manually curated demonstrations. A representative starting point in this line is Toolformer [121], which showed that a language model could annotate its own training corpus with API calls and corresponding outputs, thus learning when and how external tools should be invoked with minimal human supervision. More concretely, the key idea is to retain a candidate tool call only when its returned result improves next-token prediction under the base language modeling objective. This can be written as the reduction in prediction loss after inserting the tool response:

$$\Delta \ell_i(c, r) = \ell_i^- - \ell_i^+ = -\log p_\theta(x_i | c) + \log p_\theta(x_i | c, r), \quad (1)$$

where c denotes the original context, r denotes the tool response, and x_i denotes a subsequent token to be predicted. This criterion captures the central intuition of self-supervised tool annotation: a tool call is useful not merely because it is syntactically valid, but because it measurably improves the model’s prediction of what comes next.

Subsequent work extended this idea from local self-annotation to richer synthetic trajectory generation. ToolAlpaca [133] used synthetic tool-use demonstrations generated by stronger models to train smaller open-source LLMs,

showing that instruction-tuned tool-use behavior could be effectively transferred through generated corpora. AutoAct [110] further reduced reliance on handcrafted demonstrations by automatically synthesizing planning trajectories for tool-augmented agents, making it possible to bootstrap agentic behavior from limited initial supervision.

As the focus shifted from trajectory generation to function-calling quality, later work increasingly emphasized verifiability and data quality control. APIGen [81] introduced a multi-stage pipeline for generating and validating API-call training data, improving the reliability of synthetic supervision through executable verification. ToolACE [79] further developed this direction with a self-evolving framework that combines API augmentation, dialogue synthesis, and hierarchical validation to produce higher-quality tool-use data. Confucius [38] complements these efforts by incorporating introspective feedback and curriculum-style refinement, highlighting that synthetic data generation can be improved not only by scale, but also by progressively structured supervision. These methods show that self-supervised and synthetic data generation provides a practical foundation for training tool-using models at scale, especially when manual collection of high-quality trajectories is expensive or infeasible.

4.2 Large-Scale Instruction Tuning

Once synthetic and self-supervised supervision became feasible, the focus of Paradigm II shifted from data construction to scale and generalization. The goal was no longer to train agents for a small set of predefined tools, but to build models that could handle thousands of diverse real-world APIs with stronger generalization across domains and interfaces. A representative turning point in this direction is ToolLLM [112], which introduced ToolBench, a large-scale dataset covering more than 16,000 real-world RESTful APIs, and showed that instruction tuning on broad API corpora could substantially expand the scope of tool-use capability.

As tool libraries became larger and more dynamic, large-scale tuning also required better support for retrieval, adaptation, and tool selection. Gorilla [104] addressed the problem of API evolution through retrieval-aware training, enabling models to condition tool use on retrieved documentation rather than relying solely on static parametric memory. API-Bank [73] complemented this line by emphasizing the planning and retrieval abilities needed to select appropriate tools from large candidate pools. To further improve scalability, AnyTool [34] introduced hierarchical retrieval over large toolsets, while EasyTool [181] reduced context overhead by transforming lengthy documentation into more token-efficient representations.

Recent work has increasingly focused on improving invocation efficiency and generalization in practical function-calling settings. Octopus v2 [22] optimized latency through a lightweight mapping strategy for function arguments, and ToolkenGPT [43] represented tools as parametric token embeddings to support large tool collections without excessively expanding the vocabulary. For more specialized high-performance function calling, xLAM [187] targeted large action spaces, while Granite-Function-Calling [1] improved parameter prediction through multi-task learning. Functionary [88] further emphasized the tight integration of conversational interaction and function invocation, and NexusRaven [127] highlighted robust zero-shot generalization to unseen tool definitions. These methods show how large-scale instruction tuning moves beyond isolated tool-use demonstrations toward general-purpose models that can retrieve, interpret, and invoke broad tool ecosystems with increasing efficiency and reliability.

4.3 Process-Oriented and Alignment-Focused Tuning

The third sub-paradigm moves beyond the simple correctness of function calls to optimize the reasoning process that leads to tool use and to align that process with human preferences and safety requirements. Executing a tool is not sufficient if the underlying rationale is flawed, brittle, or misaligned with the intended objective. A representative early

effort in this direction is FireAct [16], which showed that fine-tuning on reasoning trajectories can better internalize problem-solving structure than direct input-output supervision alone. ToRA [41] further strengthens this line by training on interleaved reasoning-action trajectories with explicit rationales before each tool call, helping bridge the semantic gap between language reasoning and executable actions. Synapse [195] extends this idea by distilling agent trajectories into exemplars that can guide subsequent decision making.

Another line of work focuses on making these reasoning processes more robust and discriminative. Masked Thought [17] introduces a training objective that masks parts of the reasoning chain and asks the model to recover the missing logic, thereby discouraging superficial pattern matching. Agent-FLAN [26] decomposes agent capability into format following and reasoning, and uses negative samples to reduce hallucinated actions. In retrieval-dependent settings, RAFT [189] further improves robustness by training models to distinguish relevant from irrelevant retrieved documents, which is particularly useful when tool use depends on external information sources.

Alignment and safety then become increasingly important once tool-using models are deployed in realistic environments. ToolAlign [27] introduces the H2A principle, balancing helpfulness, harmlessness, and autonomy in tool-use supervision. ToolSword [177] targets safety alignment more directly by constructing adversarial scenarios such as prompt injection attacks for robust tool-use training, while RoT [178] improves stability under noisy conditions by exposing models to imperfect tool documentation during training. MetaTool [51] complements these efforts by training models to assess whether a tool is actually needed before attempting invocation. Finally, several works emphasize that tool specialization should not come at the expense of broader competence. Lemur [166] balances function calling, planning, and general language understanding within a unified framework, and AgentTuning [183] preserves generalist ability by mixing agent-oriented supervision with ordinary conversational data. These methods show that tuning for tool use must optimize not only whether a model can call tools, but also how it reasons, when it acts, and whether its behavior remains aligned, robust, and broadly capable.

5 Reward-Driven Tool Policy Learning

Compared with the previous paradigm, which mainly internalizes tool-use behavior through static supervision, this paradigm focuses on optimizing tool interaction through environmental feedback. The central challenge is no longer only whether a model can call a tool correctly, but also when it should act, how it should recover from failures, and how it can improve its strategy over multi-turn interactions. For this reason, recent research has increasingly turned to reinforcement learning, treating tool use as a sequential decision-making problem rather than a purely imitative one. To organize this paradigm, we group existing methods into three categories according to the dominant scope of policy optimization: Strategic Decision Optimization, End-to-End Policy Learning for Multi-Turn Reasoning, and Holistic and Multimodal Agentic Frameworks. As illustrated in Figure 5, these categories reflect a progression from optimizing individual tool-use decisions to training more general policies for long-horizon and multimodal agent behavior.

5.1 Strategic Decision Optimization

The earliest application of reinforcement learning in this paradigm focused on high-level strategic decisions about tool use. Rather than optimizing the entire reasoning-action trajectory, this line of work asks a more basic but important question: when should an agent invoke an external tool, and when should it rely on its internal knowledge instead? The objective is not only to improve accuracy, but also to balance tool effectiveness against computational cost and latency. In practice, such strategic tool-use policies are often optimized with policy-gradient style reinforcement learning

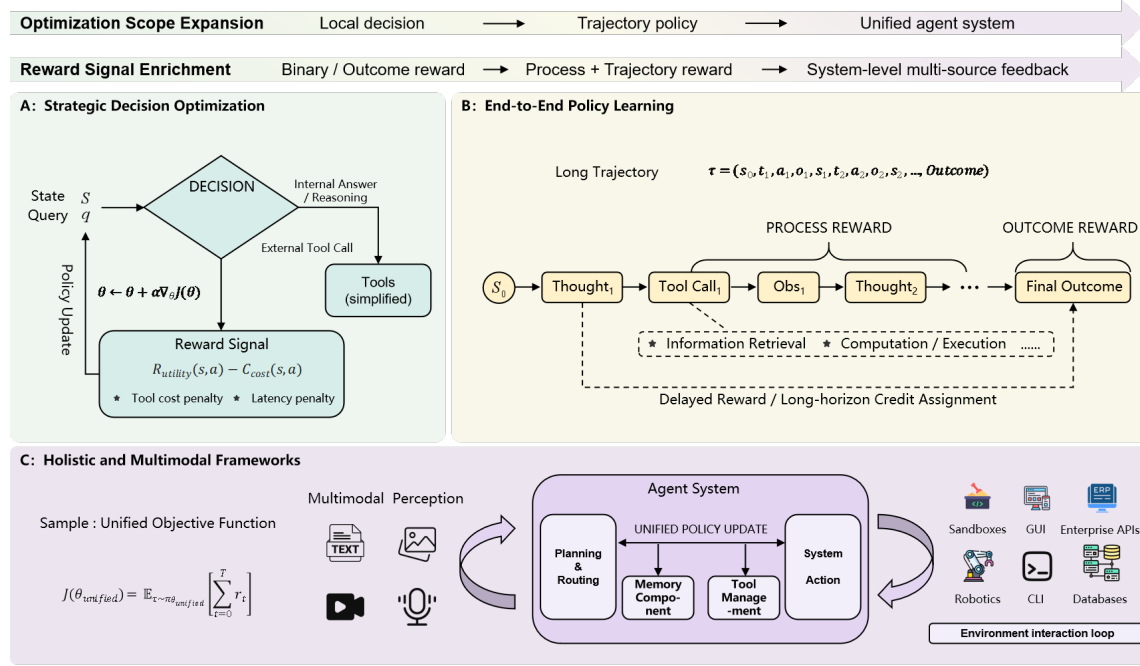


Fig. 5. An overview of Paradigm III: Reward-Driven Tool Policy Learning.

objectives. A widely used method is the PPO clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (2)$$

where

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (3)$$

Here, $\hat{A}_t$ denotes the estimated advantage of a tool-use action, and the clipping operation stabilizes policy updates by preventing overly large deviations from the previous policy. In the context of strategic decision optimization, it captures how an agent can gradually learn whether invoking a tool at a given state is beneficial under reward feedback.

A representative starting point in this direction is ReTool [35], which formulates tool invocation as a reinforcement learning problem and focuses on learning when external tools should be used during reasoning. Rather than assuming that more tool use is always beneficial, it optimizes a reward signal that favors correct outcomes while discouraging unnecessary calls, thereby improving the selectivity and efficiency of tool-assisted reasoning. ToolRL [107] further simplifies this setting by showing that carefully scaled outcome-based rewards can already provide an effective signal for optimizing tool-use behavior, reducing the need for more elaborate reward engineering.

Subsequent work extended strategic optimization toward capability expansion and self-improvement. ToolExpander [19] introduces a dynamic multi-round hard-sampling strategy that identifies persistent failure cases and replaces them with stronger demonstrations during reinforcement learning, effectively serving as a curriculum for weaker models. Agent0 [160] moves further in this direction by allowing the agent to act as both curriculum designer and executor, using Ambiguity Dynamic Policy Optimization (ADPO) to generate tasks near the frontier of its current

ability and improve without external data. In more specialized settings, Arcee [144] extends this idea to domain-specific workflows through a Large Action Model trained with reinforcement learning, showing that strategic policy optimization can also support structured enterprise action spaces. These methods focus less on full end-to-end autonomy than on improving high-level decisions about whether, when, and how agents should act.

5.2 End-to-End Policy Learning for Multi-Turn Reasoning

Compared with strategic decision optimization, this line of work directly optimizes the full reasoning-action policy over multi-turn interactions. The key challenge is no longer only whether an agent should invoke a tool, but how to assign credit across long trajectories where an early mistake may propagate through later reasoning and execution steps. SearchR1 [59] is a representative example in this direction, showing that large-scale reinforcement learning can train agents to develop search-oriented reasoning strategies such as query reformulation and information synthesis from sparse outcome rewards alone. SimpleTIR [167] further pushes this setting toward end-to-end optimization by jointly training reasoning and tool execution within a unified reinforcement learning loop, reducing the mismatch introduced by pipeline-style designs. Earlier efforts such as Retroformer [175] and TRICE [109] provide important foundations for this line by enabling agents to learn from retrospective execution feedback and by improving credit assignment in long tool-use chains.

A second line of research seeks to make this optimization more stable by introducing denser or more structured learning signals. AgentPRM [158] adapts process reward modeling to agentic tasks, evaluating intermediate steps in terms of both their promise and their progress toward the goal. LeTS [188] similarly combines process-level and outcome-level rewards to guide the joint development of reasoning and retrieval behavior. RAGEN [151] extends this perspective to self-evolving agents by generating reflective revision data from prior trajectories and using it to reinforce error-correcting behaviors during multi-turn reinforcement learning.

More recent work explores richer coordination mechanisms within end-to-end policy learning. MATPO [91] introduces role-aware optimization for planner-worker coordination, distributing rewards across distinct functions within the same agent framework so that delegation and execution can be improved simultaneously. Agent-R [180] further treats reflection as an explicit action inside the reinforcement learning loop, allowing the agent to critique past behavior and incorporate those critiques into future policy updates. In this setting, reinforcement learning is used not merely to refine isolated tool decisions, but to shape how agents search, recover, coordinate, and revise their behavior over extended interaction horizons.

5.3 Holistic and Multimodal Agentic Frameworks

The frontier of Paradigm III is expanding beyond text-based APIs to encompass multimodal interactions and holistic system optimization. As agents are deployed in rich visual environments, the definition of tool use evolves to include visual perception, GUI manipulation, and long-term memory management.

VerlTool [56] proposes a holistic framework for agentic reinforcement learning, arguing that optimizing tool calls in isolation is insufficient. It treats the entire agent system, including its memory module and tool selector, as a unified policy and updates all components jointly. DeepAgent [74] complements this perspective by introducing an autonomous memory folding mechanism that compresses past interactions into structured episodic memory to maintain coherence over long horizons. AgentRL [186] extends this line toward scalable multi-turn and multi-task training through an asynchronous generation-training pipeline, a unified function-call based API interface, and a centralized environment controller, highlighting the growing importance of system-level infrastructure in agentic reinforcement learning.

RLFactory [15] further supports this trend by providing a plug-and-play infrastructure for simulating tool environments and computing trajectory-level rewards, facilitating the development of self-evolving agents.

In the domain of visual and embodied agents, Agent Q [106] combines guided Monte Carlo Tree Search (MCTS) with a self-critique mechanism, allowing agents to learn from both successful and unsuccessful trajectories in interactive web environments through self-play. SEAgent [130] focuses on computer-use agents that learn from experience, employing a self-evolution mechanism in which the agent autonomously explores the OS environment and curates its own instruction-tuning data. DigiRL [6] addresses the challenge of training in-the-wild device control agents by introducing a scalable offline-to-online reinforcement learning framework that allows agents to refine their GUI interaction policies through autonomous exploration on Android devices.

Advanced frameworks also incorporate structural priors and multimodal grounding. GEPO [179] dynamically constructs a state-transition graph from agent experience and uses graph centrality to guide exploration toward high-impact states in open-ended environments. MGPO [50] specifically targets visual reasoning, enabling agents to refine their focus on key image regions through sequential decision-making. OS-Copilot [157] presents a framework for building generalist computer agents that optimize user intent satisfaction across diverse applications. Cradle [132] extends this idea to general computer control, enabling agents to operate software through screen observation and keyboard-mouse emulation. ToolRM [2] complements these frameworks by introducing specialized outcome reward models for evaluating the efficiency and correctness of tool-use trajectories, providing stronger learning signals for complex agentic reinforcement learning.

6 Evaluating Agentic Tool Use: Benchmarks and Metrics

The evaluation of agentic tool use has evolved alongside the underlying methodological paradigms. Early benchmarks focused on whether models could generate valid and schema-compliant tool calls, capturing the correctness of tool invocation at the function level. As tool use became more tightly integrated with reasoning and planning, evaluation expanded to end-to-end task success on static datasets. More recent benchmarks further move toward interactive and high-fidelity environments, where agents must adapt to changing states, recover from execution errors, and satisfy safety constraints. Following this progression, we organize existing evaluation benchmarks into three levels: tool usage correctness, task completion, and tool-driven interaction.

6.1 Tool Usage Correctness

This category forms the diagnostic foundation of agentic tool-use evaluation. It asks whether a model can understand tool specifications, determine whether tool use is necessary, select the appropriate function from a candidate pool, and produce structurally valid calls with correctly grounded arguments. In contrast to the later layers of evaluation, benchmarks at this level primarily assess local competence in tool invocation rather than end-to-end task success. Representative benchmarks in this layer are summarized in Table 2.

(1) Function-call validity. A first line of work evaluates whether a model can produce structurally correct tool invocations once the relevant function is available. The Berkeley Function Calling Leaderboard (BFCL) [103] has become a widely used reference point for this setting, emphasizing function selection, argument filling, and schema-constrained generation across diverse calling formats. However, aggregate pass or fail scores often obscure where the failure actually occurs. ToolEyes [176] therefore introduces a more fine-grained diagnostic framework that separates errors in intent understanding, format alignment, and tool selection. T-Eval [25] pushes diagnosis further by decomposing tool use into

Table 2. Representative benchmarks for tool usage correctness.

Year	Benchmark	Primary Focus	Executable	Compositional	Tool Source
Category I: Function-Call Validity					
2025	BFCL [103]	Function-call validity	✗	✗	Curated functions
2025	ToolEyes [176]	Fine-grained error diagnosis	✗	✗	Curated functions
2024	T-Eval [25]	Step-wise invocation correctness	✗	✓	Curated functions
Category II: Tool Retrieval and Selection					
2023	API-Bank [73]	API retrieval and selection	✓	✓	Simulated tools
2024	StableToolBench [42]	Large-scale tool selection	✓	✓	Real-world APIs
2024	Gorilla [104]	Retrieval over evolving APIs	✓	✗	Evolving API docs
Category III: Tool-Use Decisions					
2025	When2Call [119]	Tool-use necessity	✗	✗	Curated functions
2024	WTU-Eval [93]	Whether-or-not tool usage	✗	✗	Curated functions
Category IV: Compositional Invocation					
2023	NexusRaven [127]	Complex function calling	✗	✓	Curated functions
2025	NESTFUL [7]	Nested API calls	✓	✓	Curated APIs

intermediate steps, enabling evaluation of reasoning and execution decisions along the trajectory rather than only at the final call.

(2) Tool Retrieval and Selection. As the number of available tools grows, correctness is no longer limited to schema compliance, but also depends on whether the model can retrieve and select the right tool from a large candidate set. API-Bank [73] provides an early benchmark for this broader pipeline, covering API search, planning, and response generation in simulated tool-use scenarios. StableToolBench [42] extends this line toward more reliable large-scale benchmarking, improving the reproducibility of evaluation over real-world APIs. Gorilla [104] complements these benchmarks by stressing adaptation to evolving documentation, thereby testing whether models can maintain correct tool use when API specifications change over time.

(3) Tool-Use Decisions. An equally important aspect of correctness is deciding whether tool use is appropriate at all. Many earlier benchmarks implicitly assume that a tool should be called, but realistic agents must also decide when to answer directly, when to request clarification, and when to abstain. When2Call [119] directly targets this decision boundary by evaluating whether models can choose among tool invocation, follow-up questioning, and non-use. WTU-Eval [93] similarly studies whether-or-not tool usage across both tool-required and tool-optional settings, highlighting that unnecessary invocation can degrade overall performance even when the downstream tool is available.

(4) Compositional Invocation. Finally, some benchmarks focus on correctness under compositional call structures, where invocation quality depends on handling more complex signatures or passing outputs across calls. NexusRaven [127] emphasizes zero-shot generalization from tool descriptions and remains representative of benchmarking complex function-calling behavior without heavy task-specific tuning. NESTFUL [7] sharpens this challenge by evaluating nested sequences of API calls, where the output of one call becomes the input of a subsequent one. These settings are still local in the sense that they do not yet measure full end-to-end task completion, but they better capture the compositional demands that modern agentic systems increasingly face.

Taken together, benchmarks in this layer show that tool usage correctness is a multi-faceted notion. It includes not only syntactic validity, but also tool selection under scale, calibrated decisions about when tool use is necessary, and compositional accuracy in more complex invocation patterns. These diagnostic benchmarks therefore provide the

foundation for higher-level evaluation, where the focus shifts from local invocation quality to complete task execution and sustained interaction.

6.2 Task Completion

This category evaluates whether an agent can successfully complete an end-to-end task when tools are used as functional components of the solution pipeline, rather than merely producing a syntactically valid call. In contrast to diagnostic correctness benchmarks, task-completion evaluations emphasize final-task success under realistic inputs, domain constraints, and execution feedback, thereby capturing both reasoning quality and tool-mediated effectiveness. Representative benchmarks in this layer are summarized in Table 3.

Table 3. Representative benchmarks for task completion.

Year	Benchmark	Task Type	Tool Role	Executable	Open-Ended
Category I: Mathematical and Knowledge-Centric Reasoning Tasks					
2025	AIME 25 [87]	Competition math	Compute	✗	✗
2023	TabMWP [83]	Table math reasoning	Compute	✗	✗
2023	FreshQA [140]	Knowledge QA	Retrieve	✗	✗
2023	ToolQA [201]	Tool-augmented QA	Retrieve + compute	✓	✗
2021	MATH / GSM8K [29, 46]	Foundation math	Compute	✗	✗
2018	HotpotQA [171]	Multi-hop QA	Retrieve	✗	✗
Category II: Programming and Data-Centric Task Execution					
2025	BigCodeBench [202]	Code generation	Execute	✓	✓
2024	SWE-bench [58]	Software engineering	Execute + debug	✓	✓
2024	BIRD / Spider 2.0 [69, 72]	Text-to-SQL	Query	✓	✓
2024	DSBench [60]	Data science	Execute	✓	✓
2024	InterCode [168]	Interactive coding	Execute + debug	✓	✓
2024	SheetBench [85]	Spreadsheet tasks	Execute	✓	✓
2021	HumanEval / MBPP [5, 21]	Python coding	Execute	✓	✓
Category III: Scientific and Professional Domain Benchmarks					
2024	SciBench [146]	Scientific reasoning	Compute	✗	✗
2024	SciCode [136]	Scientific coding	Execute	✓	✓
2024	BioPlanner [96]	Biological protocols	Plan + retrieve	✗	✓
2024	MedAgent-Bench [57]	Clinical tasks	Plan + retrieve	✗	✓
2023	ChemCrow [13]	Chemistry workflows	Plan + execute	✓	✓

(1) Mathematical and Knowledge-Centric Reasoning Tasks. A major line of task-completion evaluation studies whether agents can solve reasoning problems more reliably when equipped with calculators, code interpreters, search engines, or other external tools. MATH [46] and GSM8K [29] are widely reused as foundational testbeds for measuring whether tool use improves multi-step numerical reasoning, while AIME [87] and MATH-500 [77] further raise the difficulty to competition-level mathematics. TabMWP [83] extends this setting to table-grounded mathematical reasoning, where solving the task often requires parsing structured inputs and executing formula-based computations. On the knowledge side, HotpotQA [171] remains a representative benchmark for multi-hop retrieval and answer composition, and ToolQA [201] more directly evaluates end-to-end question answering with external tools as functional components of the solution process. FreshQA [140] complements these benchmarks by focusing on fast-changing world knowledge, thereby testing whether agents can complete information-seeking tasks that static parametric knowledge alone cannot support. More general QA benchmarks such as 2WikiMultiHopQA [47], Natural Questions [67], TriviaQA [61], MMLU [45], and StrategyQA [39] are also frequently reused to study whether retrieval or tool augmentation improves final answer quality.

(2) Programming and Data-Centric Task Execution. A second line of evaluation focuses on whether agents can complete programming, database, and data-analysis tasks under executable feedback. HumanEval [21] and MBPP [5] provide the classical starting point for execution-based code generation, while BigCodeBench [202] substantially increases realism by requiring library-intensive code generation across a large set of APIs. Beyond function-level synthesis, SWE-bench [58] evaluates repository-level software engineering by asking agents to resolve real-world GitHub issues in existing codebases, making it especially relevant for end-to-end agentic coding. BIRD [72] and Spider 2.0 [69] extend task completion to enterprise-scale text-to-SQL settings, where success depends on schema understanding, planning, and executable query generation. DSBench [60] and DS-1000 [68] further broaden this space to realistic data-science workflows, from data cleaning to model development. In more specialized settings, FinQA [24] targets numerical reasoning over financial documents, SheetBench [85] addresses spreadsheet manipulation, and InterCode [168] standardizes interactive coding with execution feedback, allowing agents to debug and refine scripts during task solving.

(3) Scientific and Professional Domain Benchmarks. Task-completion evaluation has also expanded into scientific and professional domains, where tool use is required to satisfy domain-specific constraints rather than only to improve generic reasoning. SciBench [146] tests whether agents can solve physics and chemistry problems that often require equation handling, unit-aware reasoning, and code-based computation. SciCode [136] moves toward code-driven scientific problem solving in areas such as physics and materials science. BioPlanner [96] evaluates the generation of biological protocols, emphasizing structured procedural planning and precise function retrieval. ChemCrow [13] and MedAgent-Bench [57] further demonstrate that task-completion benchmarks now cover high-stakes workflows such as organic synthesis and clinical diagnosis, where the agent must combine reasoning, specialized tools, and domain knowledge to produce actionable solutions.

6.3 Tool-Driven Interaction

This category evaluates agents in dynamic and stateful environments where tool use directly affects the external world. Unlike static QA or offline execution benchmarks, these settings require agents to update plans as the environment changes, recover from intermediate errors, and satisfy user goals under realistic constraints. The focus therefore shifts from whether a tool call is correct in isolation to whether a sequence of actions can reliably achieve a goal in an interactive setting. Table 4 summarizes representative benchmarks in this layer.

(1) Web Environments. WebArena [200] and VisualWebArena [65] establish the core benchmark line for interactive web agents by providing high-fidelity environments in which agents must navigate functional websites through textual and visual observations. WebVoyager [44] extends this setting to open-ended web tasks on real websites, requiring more robust planning and multimodal grounding. Mind2Web [30] complements these benchmarks with offline trajectories over diverse DOM structures, making it useful for evaluating generalization before full online deployment. Beyond browser navigation, GAIA [90] evaluates general assistant style tasks that require tool use, web access, and multi-step problem solving, while TravelPlanner [161] focuses on constrained planning with external search tools. WorldSense [9] further broadens interaction evaluation to grounded 3D environments, where the agent must maintain a consistent world model while acting.

(2) Computer Environments. A second line of work evaluates agents that interact with operating systems, applications, and workplace software. OS-Copilot [157] and OSWorld [163] benchmark agents on desktop control through terminal commands, GUI actions, and multimodal perception across realistic computer tasks. OmniACT [62], ScreenAgent [95], and GUI-World [18] further emphasize grounded interaction with graphical interfaces, while AndroidWorld [117],

Table 4. Evaluation benchmarks for tool-driven interaction.

Year	Benchmark	Trajectory	Target	Challenge	Signal
Web Environments					
2024	WebArena [200]	✓	Web	Navigation	Task success
2024	VisualWebArena [65]	✓	Web	Multimodal grounding	Task success
2024	WebVoyager [44]	✓	Web	Open-ended tasks	Task success
2024	GAIA [90]	✓	General assistant	Multi-step tool use	Task success
2024	TravelPlanner [161]	✓	Planning	Constraint satisfaction	Task success
2023	WorldSense [9]	✓	3D environment	World-model reasoning	Task success
2023	Mind2Web [30]	✗	Web trajectories	Offline generalization	Trajectory match
Computer Environments					
2024	OSWorld [163]	✓	Desktop / OS	Long-horizon control	Task success
2024	OS-Copilot [157]	✓	Desktop / OS	Console and GUI control	Task success
2024	OmniACT [62]	✓	Cross-app workflows	Scripted interaction	Task success
2025	AndroidWorld [117]	✓	Mobile / Android	App interaction	Task success
2025	AppWorld / AppAgent [138, 185]	✓	Mobile apps	App control	Task success
2024	ScreenAgent [95]	✓	GUI	Visual UI automation	Task success
2024	GUI-World [18]	✓	GUI	Multimodal grounding	Task success
2024	OfficeBench [149]	✓	Office software	Procedural workflows	Task success
2024	WorkArena [33]	✓	Enterprise platform	Workflow completion	Task success
2024	τ -bench [172]	✓	User + tools	Policy compliance	State outcome
2024	TheAgentCompany [165]	✓	Workplace simulation	Consequential tasks	Task success
2024	CodeAct [145]	✓	Interactive environments	Code as action	Task success
Safety and Reliability					
2024	ToolSword [177]	✗	Tool outputs	Prompt-injection defense	Attack resistance
2024	InjEcT-Agent [184]	✗	Tool outputs	Indirect prompt injection	Attack resistance
2024	ToolEmu [120]	✓	Tool sandbox	Risk-aware interaction	Failure analysis
2024	SafetyBench [191]	✗	Safety scenarios	Risk refusal	Safety judgment
2024	R-Judge [182]	✗	Risk evaluation	Risk awareness	Risk judgment
2024	PsySafe [192]	✗	Psychological safety	Manipulation resistance	Safety judgment

AppWorld [138], and AppAgent [185] extend this line to mobile environments. In workplace settings, OfficeBench [149] and WorkArena [33] evaluate interaction with enterprise platforms and office software, where task success depends on long-horizon state tracking and procedural correctness. Recent benchmarks move further toward realistic digital work. τ -bench [172] evaluates tool-agent-user interaction under domain policies and database state constraints, while TheAgentCompany [165] benchmarks agents on consequential workplace tasks that combine web browsing, code execution, and collaboration in a simulated company environment. CodeAct [145] provides a complementary perspective by treating executable code as a unified action space for interactive environments.

(3) Safety and Reliability. As agents become more capable of acting in external environments, evaluation must also examine whether they remain reliable and safe under adversarial or high-risk conditions. ToolSword [177] and InjEcT-Agent [184] focus on indirect prompt injection, testing whether malicious instructions embedded in tool outputs can hijack subsequent agent behavior. ToolEmu [120] addresses this problem from a complementary angle by using an LLM-simulated sandbox to emulate high-risk tool execution and expose dangerous failure modes without causing real-world harm. R-Judge [182] and SafetyBench [191] evaluate whether agents can identify and refuse risky actions, while PsySafe [192] expands this line to psychological safety and manipulation resistance. Together, these benchmarks show that interaction-level evaluation is not only about completing tasks, but also about maintaining reliability, policy compliance, and safe behavior when actions have real consequences.

7 Future Directions

As the paradigm of agentic tool use matures from experimental prototypes to production grade systems, we anticipate several critical trajectories that will define the next generation of autonomous agents. These directions move beyond

merely improving the accuracy of API calls, aiming instead to reshape the fundamental architecture, interactivity, and societal integration of AI agents.

(1) **Standardization and the MCP Revolution**

The current ecosystem of tool-using agents is fragmented with bespoke integrations required for every new tool and framework. To overcome this scalability bottleneck the field is moving towards universal interoperability protocols most notably the Model Context Protocol (MCP) [49]. MCP serves as a standardized interface that decouples the agent from the specific implementation of tools allowing developers to build a tool once and have it be instantly usable by any compliant agent. This standardization transforms the integration problem from a quadratic complexity into a linear one. Beyond protocols we see a shift towards standardized orchestration patterns. OpenAI Swarm [116] introduces ergonomic patterns for multi-agent orchestration exploring lightweight handoffs between specialized agents while AgentKit [156] provides a unified framework for "reinforcement fine-tuning" of agentic workflows. We foresee a future where tool libraries are not static lists but dynamic protocol-compliant marketplaces that agents can query and subscribe to in real-time.

(2) **Multimodal and Embodied Foundation Models**

Future agents will transcend the limitations of text-based APIs to interact with the world through rich multimodal interfaces. Rather than relying solely on structured function calls agents are evolving to perceive and manipulate Graphical User Interfaces (GUIs) directly. A pivotal shift in 2025 is the emergence of "Agentic Foundation Models" specifically pre-trained for action. Magma [169] represents this new breed a multimodal model capable of interpreting and grounding inputs across both digital and physical environments effectively bridging the gap between UI navigation and robotic manipulation. Complementing this ShowUI [78] introduces UI-guided visual token selection to handle high-resolution screens efficiently while OmniParser [84] provides a pure vision-based screen parsing module that extracts structured elements from screenshots significantly enhancing the action prediction capabilities of generalist models. Furthermore Agent-S [3] establishes an open agentic framework that uses computers like a human learning from experience to perform complex cross-application tasks. These advancements signal a move away from patching LLMs with vision encoders towards building native "Vision-Language-Action" models.

(3) **Agentic Self-Consistency and Continuous Evolution**

To achieve true autonomy agents must move beyond reactive single-turn execution towards System 2 reasoning and lifetime learning. This entails the adoption of test-time scaling techniques where agents perform deep lookahead searches or self-consistency checks before executing critical tool calls. Frameworks like SearchR1 [59] have demonstrated the power of internalizing reasoning steps but future agents will go further by actively managing their memory. Titans [8] introduces a neural long-term memory module that learns to memorize historical context at test time enabling agents to handle "needle-in-a-haystack" retrieval tasks in massive interaction histories without retraining. System 2 Attention [154] enables agents to dynamically regenerate their own context to focus on relevant information while Trove [150] facilitates the continuous evolution of an agent's tool library. Instead of resetting after each session future agents will curate their own internal libraries of synthesized tools and usage patterns achieving a form of self-consistency where their actions become increasingly efficient and aligned with their historical experience. Beyond improving tool use efficiency, reinforcement learning may also optimize behavior specific properties of agents. Character-R1 suggests that future agent optimization may extend to role aware reasoning and persona consistency in specialized settings [134].

(4) Higher Safety Standards and Trustworthiness

As agents are granted the power to execute consequential actions such as modifying databases or managing financial transactions safety must be elevated from a secondary metric to a primary design constraint. Future research will focus on establishing bank-grade security protocols that go beyond simple alignment tuning. This includes developing robust defenses against indirect prompt injections that could hijack tool execution as highlighted by InjEcT-Agent [184]. Industrial solutions like NeMo Guardrails [118] are beginning to provide programmable safety layers that enforce behavioral boundaries while PromptArmor [124] offers detection mechanisms for adversarial inputs. Crucially we must address the risk of deceptive alignment ensuring that agents do not learn to game reward systems by hiding errors or misrepresenting their tool usage a phenomenon investigated in Sleeper Agents [52]. Trustworthiness will rely on verifiable audit trails and sandboxed execution environments that guarantee human control even as agent autonomy increases.

(5) Human-Agent Symbiosis and Workflow Reshaping

A final direction is human-agent symbiosis and workflow reshaping. As systems such as MetaGPT [48], AutoGen [155], and OpenDevin [147] show, the future of tool use may lie less in fully autonomous monolithic agents than in collaborative workflows that combine human oversight with specialized automated components. Relatedly, recent work on LLM based multi agent systems suggests that future agentic workflows may increasingly rely on system level coordination, role allocation, and orchestration across multiple specialized agents rather than a single monolithic agent [198]. In this setting, skills may become a more practical abstraction than end-to-end autonomy: instead of requiring a single agent to solve every task from start to finish, systems can expose reusable capabilities such as retrieval, coding, spreadsheet editing, verification, or domain-specific analysis, and compose them flexibly under human supervision. This skill-centric view also helps explain why simpler and more structured systems can remain highly competitive. Agentless [159], for example, suggests that carefully designed pipelines may outperform more complex long-horizon agents when task structure is clear. Together, these trends point toward a future in which agentic tool use is shaped by calibrated delegation, reusable skills, and workflow-aware collaboration between humans and agents.

8 Conclusion

In this survey, we have systematically reviewed the technological landscape of agentic tool use, examining the transition from prompting to dynamic autonomous optimization. We categorized this domain into three distinct paradigms: Paradigm I leverages prompting and context-based strategies to enable tool use in a plug-and-play fashion; Paradigm II uses supervised learning to internalize robust tool-use capabilities; and Paradigm III uses reward-driven policy learning to enable agents to adapt, self-correct, and recover from complex execution errors. Alongside these paradigms, we treated evaluation as a cross-cutting dimension and surveyed the evolving benchmark ecosystem, which has progressed from evaluating isolated function call correctness to measuring holistic task success and safety in realistic simulations. We hope this survey provides a coherent roadmap for researchers and practitioners, and helps accelerate the development of tool-using agents towards being more capable, reliable, and standardized autonomous agents.

References

- [1] Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, Sadhana Kumaravel, Matthew Stallone, Rameswar Panda, Yara Rizk, G P Shrivatsa Bhargav, Maxwell Crouse, Chulaka Gunasekara, Shajith Ikbal, Sachindra Joshi, Hima Karanam, Vineet Kumar, Asim Munawar, Sumit Neelam, Dinesh Raghu, Udit Sharma, Adriana Meza Soria, Dheeraj Sreedhar, Praveen Venkateswaran, Merve Unuvar, David Daniel Cox, Salim Roukos, Luis A. Lastras, and Pavan Kapanipathi. 2024. Granite-Function Calling Model: Introducing Function Calling Abilities via Multi-Task Learning of Granular Tasks. In

Manuscript submitted to ACM

- Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Association for Computational Linguistics. doi:10.18653/v1/2024.emnlp-industry.85
- [2] Mayank Agarwal, Ibrahim Abdelaziz, Kinjal Basu, Merve Unuvar, Luis A. Lastras, Yara Rizk, and Pavan Kapanipathi. 2025. ToolRM: Outcome Reward Models for Tool-Calling Large Language Models. *CoRR* abs/2509.11963 (2025). arXiv:2509.11963 doi:10.48550/arXiv.2509.11963
 - [3] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. 2025. Agent S: An Open Agentic Framework that Uses Computers Like a Human. In *Proceedings of the International Conference on Learning Representations*. arXiv:2410.08164 [cs.AI] doi:10.48550/arXiv.2410.08164
 - [4] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Cristian Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Ruben Ruano, Kai Jeffrey, Sammie Jesmonth, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Keunhong Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jay Quiambao, Kanishka Rao, Jake Rettinghouse, Diego Reyes, Pierre Sermanet, Nicholas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Zeng Andy. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *CoRR* abs/2204.01691 (2022). doi:10.48550/arXiv.2204.01691
 - [5] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021). doi:10.48550/arXiv.2108.07732
 - [6] Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. 2024. DigiRL: Training In-The-Wild Device-Control Agents with Autonomous Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 37. 12461–12495. doi:10.52202/079017-0397
 - [7] Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, Xin Wang, Luis A. Lastras, and Pavan Kapanipathi. 2025. NESTFUL: A Benchmark for Evaluating LLMs on Nested Sequences of API Calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 33538–33547. doi:10.18653/v1/2025.emnlp-main.1702
 - [8] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. 2025. Titans: Learning to Memorize at Test Time. *CoRR* abs/2501.00663 (2025). doi:10.48550/ARXIV.2501.00663
 - [9] Youssef Benchekroun, Megi Dervishi, Mark Ibrahim, Jean-Baptiste Gaya, Xavier Martinet, Grégoire Mialon, Thomas Scialom, Emmanuel Dupoux, Dieuwke Hupkes, and Pascal Vincent. 2023. WorldSense: A Synthetic Benchmark for Grounded Reasoning in Large Language Models. *arXiv preprint arXiv:2311.15930* (2023). doi:10.48550/arXiv.2311.15930
 - [10] Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FACCT '21)*. Association for Computing Machinery, New York, NY, USA, 610–623. doi:10.1145/3442188.3445922
 - [11] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoeft. 2024. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*. doi:10.1609/aaai.v38i16.29720
 - [12] Christopher M. Bishop. 2006. *Pattern Recognition and Machine Learning*. Springer, New York, NY, USA. <https://link.springer.com/book/10.1007/978-0-387-45528-0>
 - [13] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. 2024. Augmenting Large Language Models with Chemistry Tools. *Nature Machine Intelligence* 6, 5 (2024), 525–535. doi:10.1038/s42256-024-00832-8
 - [14] Tianle Cai, Xuezhi Wang, Yungyue Ma, Xinyun Chen, and Denny Zhou. 2023. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2305.17126
 - [15] Jiajun Chai, Guojun Yin, Zekun Xu, Chuhan Yue, Yi Jia, Siyu Xia, Xiaohan Wang, Jiwen Jiang, Xiaoguang Li, Chengqi Dong, Hang He, and Wei Lin. 2025. RLFactory: A Plug-and-Play Reinforcement Learning Post-Training Framework for LLM Multi-Turn Tool-Use. *CoRR* abs/2509.06980 (2025). doi:10.48550/arXiv.2509.06980
 - [16] Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. 2023. FireAct: Toward Language Agent Fine-tuning. *CoRR* abs/2310.05915 (2023). doi:10.48550/arXiv.2310.05915
 - [17] Changyu Chen, Xiting Wang, Ting-En Lin, Ang Lv, Yuchuan Wu, Xin Gao, Ji-Rong Wen, Rui Yan, and Yongbin Li. 2024. Masked Thought: Simply Masking Partial Reasoning Steps Can Improve Mathematical Reasoning Learning of Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 5872–5900. doi:10.18653/v1/2024.acl-long.320
 - [18] Dongping Chen, Yue Huang, Siyuan Wu, Jingyu Tang, Liuyi Chen, Yilin Bai, Zhigang He, Chenlong Wang, Huichi Zhou, Yiqiang Li, Tianshuo Zhou, Yue Yu, Chujie Gao, Qihui Zhang, Yi Gui, Zhen Li, Yao Wan, Pan Zhou, Jianfeng Gao, and Lichao Sun. 2024. GUI-World: A Video Benchmark and Dataset for Multimodal GUI-oriented Understanding. *arXiv preprint arXiv:2406.10819* (2024). doi:10.48550/arXiv.2406.10819
 - [19] Fu Chen, Peng Wang, Xiyin Li, Wen Li, Shichi Lei, and Dongdong Xiang. 2025. ToolExpander: Extending the Frontiers of Tool-Using Reinforcement Learning to Weak LLMs. arXiv:2510.07737 doi:10.48550/arXiv.2510.07737
 - [20] Jinyang Chen, Haolun Wu, Jianhong Pang, Yihua Wang, Dell Zhang, and Changzhi Sun. 2025. Tool learning with language models: a comprehensive survey of methods, pipelines, and benchmarks. *ViciniEarth* 2 (2025), 16. doi:10.1007/s44336-025-00024-x
 - [21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie

- Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Chris Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). doi:10.48550/arXiv.2107.03374
- [22] Wei Chen and Zhiyuan Li. 2024. Octopus v2: On-Device Language Model for Super Agent. *arXiv preprint arXiv:2404.01744* (2024). doi:10.48550/arXiv.2404.01744
- [23] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research* 2023 (2023). doi:10.48550/arXiv.2211.12588
- [24] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A Dataset of Numerical Reasoning over Financial Data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 3697–3711. doi:10.18653/v1/2021.emnlp-main.300
- [25] Zehui Chen, Weihua Du, Wenwei Zhang, Kuikun Liu, Jiangning Liu, Miao Zheng, Jingming Zhuo, Songyang Zhang, Dahua Lin, Kai Chen, and Feng Zhao. 2024. T-Eval: Evaluating the Tool Utilization Capability of Large Language Models Step by Step. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 9510–9529. doi:10.18653/v1/2024.acl-long.515
- [26] Zehui Chen, Kuikun Liu, Qiuchen Wang, Wenwei Zhang, Jiangning Liu, Dahua Lin, Kai Chen, and Feng Zhao. 2024. Agent-FLAN: Designing Data and Methods of Effective Agent Tuning for Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 9354–9366. doi:10.18653/v1/2024.findings-acl.557
- [27] ZhiYuan Chen, Shiqi Shen, Guangyao Shen, Gong Zhi, Xu Chen, and Yankai Lin. 2024. Towards Tool Use Alignment of Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 1382–1400. doi:10.18653/v1/2024.emnlp-main.82
- [28] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. 2023. Binding Language Models in Symbolic Languages. In *Proceedings of the Eleventh International Conference on Learning Representations*. doi:10.48550/arXiv.2210.02875
- [29] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021). doi:10.48550/arXiv.2110.14168
- [30] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. *CoRR* abs/2306.06070 (2023). doi:10.48550/arXiv.2306.06070
- [31] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. doi:10.18653/v1/N19-1423
- [32] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. 2024. Chain-of-Verification Reduces Hallucination in Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 3563–3578. doi:10.18653/v1/2024.findings-acl.212
- [33] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. 2024. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks?. In *Proceedings of the 41st International Conference on Machine Learning*. 11642–11662. doi:10.48550/arXiv.2403.07718
- [34] Yu Du, Fangyun Wei, and Hongyang Zhang. 2024. AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls. *CoRR* abs/2402.04253 (2024). doi:10.48550/arXiv.2402.04253
- [35] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjuan Zhong. 2025. ReTool: Reinforcement Learning for Strategic Tool Use in LLMs. *CoRR* abs/2504.11536 (2025). doi:10.48550/arXiv.2504.11536
- [36] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2021. The Pile: An 800GB Dataset of Diverse Text for Language Modeling. *arXiv preprint arXiv:2101.00027* (2021). doi:10.48550/arXiv.2101.00027
- [37] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-Aided Language Models. In *Proceedings of the 40th International Conference on Machine Learning*. PMLR, 10764–10799. doi:10.48550/arXiv.2211.10435
- [38] Shen Gao, Zhengliang Shi, Minghang Zhu, Bowen Fang, Xin Xin, Pengjie Ren, Zhumin Chen, Jun Ma, and Zhaochun Ren. 2024. Confucius: Iterative Tool Learning from Introspection Feedback by Easy-to-Difficult Curriculum. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18030–18038. doi:10.1609/aaai.v38i16.29759
- [39] Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. 2021. Did Aristotle Use a Laptop? A Question Answering Benchmark with Implicit Reasoning Strategies. *Transactions of the Association for Computational Linguistics* 9 (2021), 346–361. doi:10.1162/tacal_a_00370
- [40] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *The Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2305.11738

- [41] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024. ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving. In *Proceedings of the Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2309.17452
- [42] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 11143–11156. doi:10.18653/v1/2024.findings-acl.664
- [43] Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2023. ToolkenGPT: Augmenting Frozen Language Models with Massive Tools via Tool Embeddings. *CoRR abs/2305.11554*. doi:10.48550/arXiv.2305.11554
- [44] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 6864–6890. doi:10.18653/v1/2024.acl-long.371
- [45] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *Proceedings of the International Conference on Learning Representations*. doi:10.48550/arXiv.2009.03300
- [46] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *arXiv preprint arXiv:2103.03874* (2021). doi:10.48550/arXiv.2103.03874
- [47] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, Barcelona, Spain (Online), 6609–6625. doi:10.18653/v1/2020.coling-main.580
- [48] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. *CoRR abs/2308.00352* (2023). doi:10.48550/arXiv.2308.00352
- [49] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *CoRR abs/2503.23278* (2025). arXiv:2503.23278 [cs.CR] doi:10.48550/arXiv.2503.23278
- [50] Xinyu Huang, Yuhao Dong, Weiwei Tian, Bo Li, Rui Feng, and Ziwei Liu. 2025. High-Resolution Visual Reasoning via Multi-Turn Grounding-Based Reinforcement Learning. *CoRR abs/2507.05920* (2025). arXiv:2507.05920 doi:10.48550/ARXIV.2507.05920
- [51] Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, Pan Zhou, Yao Wan, Neil Zhenqiang Gong, and Lichao Sun. 2024. MetaTool Benchmark for Large Language Models: Deciding Whether to Use Tools and Which to Use. In *Proceedings of the Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2310.03128
- [52] Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton Cheng, Adam Jermy, Amanda Askell, Ansh Radhakrishnan, Cem Anil, David Duvenaud, Deep Ganguli, Fazl Barez, Jack Clark, Kamal Ndousse, Kshitij Sachan, Michael Sellitto, Mrinank Sharma, Nova DasSarma, Roger Grosse, Shauna Kravec, Yuntao Bai, Zachary Witten, Marina Favaro, Jan Brauner, Holden Karnofsky, Paul Christiano, Samuel R. Bowman, Logan Graham, Jared Kaplan, Sören Mindermann, Ryan Greenblatt, Buck Shlegeris, Nicholas Schiefer, and Ethan Perez. 2024. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. *CoRR abs/2401.05566* (2024). doi:10.48550/ARXIV.2401.05566
- [53] Shima Imani, Liang Du, and Harsh Shrivastava. 2023. MathPrompter: Mathematical Reasoning using Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 5: Industry Track)*. Association for Computational Linguistics, Toronto, Canada, 37–42. doi:10.18653/v1/2023.acl-industry.4
- [54] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *Comput. Surveys* 55, 12 (2023), 1–38. doi:10.1145/3571730
- [55] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lelio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Theophile Gervet, Thibaut Lavril, Thomas Wang, Timothee Lacroix, and William El Sayed. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024). doi:10.48550/arXiv.2401.04088
- [56] Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, Tianyu Pang, and Wenhui Chen. 2025. VerlTool: Towards Holistic Agentic Reinforcement Learning with Tool Use. *CoRR abs/2509.01055* (2025). doi:10.48550/arXiv.2509.01055
- [57] Yixing Jiang, Kameron C. Black, Gloria Geng, Danny Park, James Zou, Andrew Y. Ng, and Jonathan H. Chen. 2025. MedAgentBench: A Virtual EHR Environment to Benchmark Medical LLM Agents. *NEJM AI* 2, 9, Article AIdbp2500144 (2025). doi:10.1056/AIdbp2500144
- [58] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *CoRR abs/2310.06770* (2024). doi:10.48550/arXiv.2310.06770
- [59] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. *CoRR abs/2503.09516* (2025). doi:10.48550/arXiv.2503.09516
- [60] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. 2024. DS-Bench: How Far Are Data Science Agents from Becoming Data Science Experts? *arXiv preprint arXiv:2409.07703* (2024). doi:10.48550/arXiv.2409.07703
- [61] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vancouver, Canada, 1601–1611. doi:10.18653/v1/P17-1147

- [62] Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem AlShikh, and Ruslan Salakhutdinov. 2024. OmniACT: A Dataset and Benchmark for Enabling Multimodal Generalist Autonomous Agents for Desktop and Web. In *Computer Vision – ECCV 2024 (Lecture Notes in Computer Science, Vol. 15126)*. Springer, Cham, Switzerland, 161–178. doi:10.1007/978-3-031-73113-6_10
- [63] Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, Dor Muhlgay, Noam Rozen, Erez Schwartz, Gal Shachaf, Shai Shalev-Shwartz, Amnon Shashua, and Moshe Tennenholtz. 2022. MRKL Systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. *CoRR abs/2205.00445* (2022). doi:10.48550/arXiv.2205.00445
- [64] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning*. doi:10.48550/arXiv.2312.04511
- [65] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. 2024. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 881–905. doi:10.18653/v1/2024.acl-long.50
- [66] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. In *Advances in Neural Information Processing Systems 35*. doi:10.48550/arXiv.2205.11916
- [67] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. doi:10.1162/tac1_a_00276
- [68] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-Tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. *CoRR abs/2211.11501* (2023). doi:10.48550/arXiv.2211.11501
- [69] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2024. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. *arXiv preprint arXiv:2411.07763* (2024). doi:10.48550/arXiv.2411.07763
- [70] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474. doi:10.48550/arXiv.2005.11401
- [71] Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. 2024. Chain of Code: Reasoning with a Language Model-Augmented Code Emulator. *CoRR abs/2312.04474* (2024). doi:10.48550/arXiv.2312.04474
- [72] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. *arXiv preprint arXiv:2305.03111* (2023). doi:10.48550/arXiv.2305.03111
- [73] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 3102–3116. doi:10.18653/v1/2023.emnlp-main.187
- [74] Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinyao Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, and Zhicheng Dou. 2025. DeepAgent: A General Reasoning Agent with Scalable Toolsets. *CoRR abs/2510.21618* (2025). arXiv:2510.21618 doi:10.48550/arXiv.2510.21618
- [75] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2022. Code as Policies: Language Model Programs for Embodied Control. *arXiv preprint arXiv:2209.07753* (2022). doi:10.48550/arXiv.2209.07753
- [76] Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, Yun Wang, Linjun Shou, Ming Gong, and Nan Duan. 2024. TaskMatrix.AI: Completing Tasks by Connecting Foundation Models with Millions of APIs. *Intelligent Computing* (2024). doi:10.34133/icomputing.0063
- [77] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let’s Verify Step by Step. In *Proceedings of the International Conference on Learning Representations*. doi:10.48550/arXiv.2305.20050
- [78] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2025. ShowUI: One Vision-Language-Action Model for GUI Visual Agent. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 19498–19508. doi:10.1109/CVPR52734.2025.01816
- [79] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, Zezhong Wang, Yuxian Wang, Ning Wu, Yutai Hou, Bin Wang, Chuhan Wu, Xinzhong Wang, Yong Liu, Yasheng Wang, Duyu Tang, Dandan Tu, Lifeng Shang, Xin Jiang, Ruiming Tang, Defu Lian, Qun Liu, and Enhong Chen. 2025. ToolACE: Winning the Points of LLM Function Calling. In *Proceedings of the International Conference on Learning Representations*. doi:10.48550/arXiv.2409.00920
- [80] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *Proceedings of the Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2308.03688

- [81] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh Murthy, Liangwei Yang, Silvio Savarese, Juan Carlos Nieves, Huan Wang, Shelby Heinecke, and Caiming Xiong. 2024. APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets. *CoRR* abs/2406.18518 (2024). doi:10.48550/arXiv.2406.18518
- [82] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-Play Compositional Reasoning with Large Language Models. *CoRR* abs/2304.09842 (2023). doi:10.48550/arXiv.2304.09842
- [83] Pan Lu, Liang Qiu, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, Tanmay Rajpurohit, Peter Clark, and Ashwin Kalyan. 2023. Dynamic Prompt Learning via Policy Gradient for Semi-structured Mathematical Reasoning. In *Proceedings of the International Conference on Learning Representations*. doi:10.48550/arXiv.2209.14610
- [84] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. 2024. *OmniParser for Pure Vision Based GUI Agent*. arXiv:2408.00203 [cs.CV] doi:10.48550/arXiv.2408.00203
- [85] Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. 2024. SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation. *arXiv preprint arXiv:2406.14991* (2024). doi:10.48550/arXiv.2406.14991
- [86] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*. doi:10.48550/arXiv.2303.17651
- [87] Mathematical Association of America. 2025. American Invitational Mathematics Examination (AIME). <https://maa.org/maa-invitational-competitions/>
- [88] MeetKai Inc. 2024. Functionary: A Chat Language Model for Function Calling. <https://github.com/MeetKai/functionary>. <https://github.com/MeetKai/functionary>
- [89] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. 2023. Augmented Language Models: A Survey. *Transactions on Machine Learning Research* (2023). doi:10.48550/arXiv.2302.07842
- [90] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A Benchmark for General AI Assistants. In *Proceedings of the Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2311.12983
- [91] Zhanfeng Mo, Xingxuan Li, Yuntao Chen, and Lidong Bing. 2025. *Multi-Agent Tool-Integrated Policy Optimization*. arXiv:2510.04678 doi:10.48550/arXiv.2510.04678
- [92] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2021. WebGPT: Browser-assisted Question-Answering with Human Feedback. *arXiv preprint arXiv:2112.09332* (2021). doi:10.48550/arXiv.2112.09332
- [93] Kangyun Ning, Yisong Su, Xueqiang Lv, Yuanzhe Zhang, Jian Liu, Kang Liu, and Jinan Xu. 2024. WTU-EVAL: A Whether-or-Not Tool Usage Evaluation Benchmark for Large Language Models. *CoRR* abs/2407.12823 (2024). doi:10.48550/arXiv.2407.12823
- [94] Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. 2023. Skeleton-of-Thought: Prompting Large Language Models for Efficient Parallel Generation. *arXiv preprint arXiv:2307.15337* (2023). doi:10.48550/arXiv.2307.15337
- [95] Runliang Niu, Jindong Li, Shiqi Wang, Yali Fu, Xiyu Hu, Xueyuan Leng, He Kong, Yi Chang, and Qi Wang. 2024. ScreenAgent: A Vision Language Model-driven Computer Control Agent. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 6433–6441. doi:10.24963/ijcai.2024/711
- [96] Odhran O’Donoghue, Aleksandar Shtedritski, John Ginger, Ralph Abboud, Ali Essa Ghareeb, and Samuel G. Rodrigues. 2023. BioPlanner: Automatic Evaluation of LLMs on Protocol Planning in Biology. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2676–2694. doi:10.18653/v1/2023.emnlp-main.162
- [97] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023). doi:10.48550/arXiv.2303.08774
- [98] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *CoRR* abs/2310.08560 (2023). doi:10.48550/arXiv.2310.08560
- [99] Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. 2023. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Singapore, 3806–3824. doi:10.18653/v1/2023.findings-emnlp.248
- [100] Bhargavi Paranjape, Scott M. Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. 2023. ART: Automatic Multi-Step Reasoning and Tool-Use for Large Language Models. *CoRR* abs/2303.09014 (2023). doi:10.48550/arXiv.2303.09014
- [101] Aaron Parisi, Yao Zhao, and Noah Fiedel. 2022. TALM: Tool Augmented Language Models. *CoRR* abs/2205.12255 (2022). doi:10.48550/arXiv.2205.12255
- [102] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST 2023)*. ACM, Article 108, 22 pages. doi:10.1145/3586183.3606763
- [103] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning*, Vol. 267. PMLR, Vancouver, Canada. <https://openreview.net/forum?id=2GmDdhBdDk>

- [104] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems* 37. doi:10.52202/079017-4020
- [105] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2023. Measuring and Narrowing the Compositionality Gap in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Singapore, 5687–5711. doi:10.18653/v1/2023.findings-emnlp.378
- [106] Pranav Putta, Edmund Mills, Naman Garg, Sumeet Motwani, Chelsea Finn, Divyansh Garg, and Rafael Rafailov. 2024. Agent Q: Advanced Reasoning and Learning for Autonomous AI Agents. *CoRR* abs/2408.07199 (2024). arXiv:2408.07199 doi:10.48550/arXiv.2408.07199
- [107] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiushi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. 2025. ToolRL: Reward is All Tool Learning Needs. In *Advances in Neural Information Processing Systems*. doi:10.48550/arXiv.2504.13958
- [108] Cheng Qian, Chi Han, Yi R. Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Singapore, 6922–6939. doi:10.18653/v1/2023.findings-emnlp.462
- [109] Shuofei Qiao, Honghao Gui, Chengfei Lv, Qianghui Jia, Huajun Chen, and Ningyu Zhang. 2024. Making Language Models Better Tool Learners with Execution Feedback. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Mexico City, Mexico, 3550–3568. doi:10.18653/v1/2024.naacl-long.195
- [110] Shuofei Qiao, Ningyu Zhang, Runnan Fang, Yujie Luo, Wangchunshu Zhou, Yuchen Eleanor Jiang, Chengfei Lv, and Huajun Chen. 2024. AUTOACT: Automatic Agent Learning from Scratch for QA via Self-Planning. *CoRR* abs/2401.05268 (2024). doi:10.48550/arXiv.2401.05268
- [111] Yujia Qin, Zihan Cai, Dian Jin, Lan Yan, Shihao Liang, Kunlun Zhu, Yankai Lin, Xu Han, Ning Ding, Huadong Wang, Ruobing Xie, Fanchao Qi, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2023. WebCPM: Interactive Web Search for Chinese Long-form Question Answering. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Toronto, Canada, 8968–8988. doi:10.18653/v1/2023.acl-long.499
- [112] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *CoRR* abs/2307.16789 (2023). doi:10.48550/arXiv.2307.16789
- [113] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. 2025. Tool Learning with Large Language Models: A Survey. *Frontiers of Computer Science* 19, Article 198343 (2025), 24 pages. doi:10.1007/s11704-024-40678-2
- [114] Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024). doi:10.48550/arXiv.2412.15115
- [115] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 140 (2020), 5485 – 5551. doi:10.48550/arXiv.1910.10683
- [116] Muhammad Atta Ur Rahman, Melanie Schranz, and Samira Hayat. 2025. LLM-Powered Swarms: A New Frontier or a Conceptual Stretch? *CoRR* abs/2506.14496 (2025). arXiv:2506.14496 [cs.AI] doi:10.48550/arXiv.2506.14496
- [117] Christopher Rawles, Sarah Clinckemallie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. 2025. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. In *Proceedings of the Thirteenth International Conference on Learning Representations*. doi:10.48550/arXiv.2405.14573
- [118] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 431–445. doi:10.18653/v1/2023.emnlp-demo.40
- [119] Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. 2025. When2Call: When (not) to Call Tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 3391–3409. doi:10.18653/v1/2025.naacl-long.174
- [120] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *Proceedings of the 12th International Conference on Learning Representations*. doi:10.48550/arXiv.2309.15817
- [121] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *CoRR* abs/2302.04761 (2023). doi:10.48550/arXiv.2302.04761
- [122] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and Its Friends in Hugging Face. *arXiv preprint arXiv:2303.17580* (2023). doi:10.48550/arXiv.2303.17580
- [123] Zhuocheng Shen. 2024. LLM With Tools: A Survey. *arXiv preprint arXiv:2409.18807* (2024). doi:10.48550/arXiv.2409.18807

- [124] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. 2025. PromptArmor: Simple yet Effective Prompt Injection Defenses. *CoRR* abs/2507.15219 (2025). doi:10.48550/ARXIV.2507.15219
- [125] Noah Shinn, Federico Cassano, Erez Berman, Arun Gopinath, Karthik R. Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *CoRR* abs/2303.11366 (2023). doi:10.48550/arXiv.2303.11366
- [126] Yifan Song, Weimin Xiong, Dawei Zhu, Wenhao Wu, Han Qian, Mingbo Song, Hailiang Huang, Cheng Li, Ke Wang, Rong Yao, Ye Tian, and Sujian Li. 2023. RestGPT: Connecting Large Language Models with Real-World Applications via RESTful APIs. *CoRR* abs/2306.06624 (2023). doi:10.48550/arXiv.2306.06624
- [127] Venkat Krishna Srinivasan, Zhen Dong, Banghua Zhu, Brian Yu, Hanzi Mao, Damon Mosk-Aoyama, Kurt Keutzer, Jiantao Jiao, and Jian Zhang. 2023. NexusRaven: A Commercially-Permissive Language Model for Function Calling. In *NeurIPS 2023 Workshop on Foundation Models for Decision Making*. <https://openreview.net/forum?id=5lcPe6Dqfl>
- [128] Haotian Sun, Yuchen Zhuang, Linghai Kong, Bo Dai, and Chao Zhang. 2023. AdaPlanner: Adaptive Planning from Feedback with Language Models. In *Advances in Neural Information Processing Systems*. doi:10.48550/arXiv.2305.16653
- [129] Simeng Sun, Yang Liu, Shuhang Wang, Dan Iter, Chengguang Zhu, and Mohit Iyyer. 2024. PEARL: Prompting Large Language Models to Plan and Execute Actions Over Long Documents. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2024)*. 469–486. doi:10.18653/v1/2024.eacl-long.29
- [130] Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. 2025. SEAgent: Self-Evolving Computer Use Agent with Autonomous Learning from Experience. *CoRR* abs/2508.04700 (2025). arXiv:2508.04700 doi:10.48550/arXiv.2508.04700
- [131] Didac Suris, Sachit Menon, and Carl Vondrick. 2023. ViperGPT: Visual Inference via Python Execution for Reasoning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2023)*. IEEE, 11854–11864. doi:10.1109/ICCV51070.2023.01092
- [132] Weihao Tan, Wentao Zhang, Xinrun Xu, Haochong Xia, Ziluo Ding, Boyu Li, Bohan Zhou, Junpeng Yue, Jiechuan Jiang, Yewen Li, Ruyi An, Molei Qin, Chuqiao Zong, Longtao Zheng, Yujie Wu, Xiaoqiang Chai, Yifei Bi, Tianbao Xie, Pengjie Gu, Xiyun Li, Ceyao Zhang, Long Tian, Chaojie Wang, Xinrun Wang, Börje F. Karlsson, Bo An, Shuicheng Yan, and Zongqing Lu. 2025. Cradle: Empowering Foundation Agents towards General Computer Control. In *Proceedings of the 42nd International Conference on Machine Learning*. doi:10.48550/arXiv.2403.03186
- [133] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. 2023. ToolAlpaca: Generalized Tool Learning for Language Models with 3000 Simulated Cases. *CoRR* abs/2306.05301 (2023). doi:10.48550/arXiv.2306.05301
- [134] Yihong Tang, Kehai Chen, Xuefeng Bai, Benyou Wang, Zeming Liu, Haifeng Wang, and Min Zhang. 2026. Character-R1: Enhancing Role-Aware Reasoning in Role-Playing Agents via RLVR. *arXiv preprint* arXiv:2601.04611 (2026). doi:10.48550/arXiv.2601.04611
- [135] Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, YaGuang Li, Hongrae Lee, Huaixiu Steven Zheng, Amin Ghafouri, Marcelo Menegali, Yanping Huang, Maxim Krikun, Dmitry Lepikhin, James Qin, Dehao Chen, Yuanzhong Xu, Zhifeng Chen, Adam Roberts, Maarten Bosma, Vincent Zhao, Yanqi Zhou, Chung-Ching Chang, Igor Krivokon, Will Rusch, Marc Pickett, Pranesh Srinivasan, Laichee Man, Kathleen Meier-Hellstern, Meredith Ringel Morris, Tulsee Doshi, Renelito Delos Santos, Toju Duke, Johnny Soraker, Ben Zevenbergen, Vinodkumar Prabhakaran, Mark Diaz, Ben Hutchinson, Kristen Olson, Alejandra Molina, Erin Hoffman-John, Josh Lee, Lora Aroyo, Ravi Rajakumar, Alena Butryna, Matthew Lamm, Viktoriya Kuzmina, Joe Fenton, Aaron Cohen, Rachel Bernstein, Ray Kurzweil, Blaise Agüera-Arcas, Claire Cui, Marian Croak, Ed Chi, and Quoc Le. 2022. LaMDA: Language Models for Dialog Applications. *CoRR* abs/2201.08239 (2022). doi:10.48550/arXiv.2201.08239
- [136] Minyang Tian, Luyu Gao, Shizhuo Dylan Zhang, Xinan Chen, Cunwei Fan, Xuefei Guo, Roland Haas, Pan Ji, Kittithat Krongchon, Yao Li, Shengyan Liu, Di Luo, Yutao Ma, Hao Tong, Kha Trinh, Chenyu Tian, Zihan Wang, Bohao Wu, Yanyu Xiong, Shengzhu Yin, Minhui Zhu, Kilian Lieret, Yanxin Lu, Genglin Liu, Yufeng Du, Tianhua Tao, Ofir Press, Jamie Callan, Eliu Huerta, and Hao Peng. 2024. SciCode: A Research Coding Benchmark Curated by Scientists. In *Advances in Neural Information Processing Systems*, Vol. 37. 30624–30650. doi:10.52202/079017-0963
- [137] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shrutli Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madsen Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rishi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint* arXiv:2307.09288 (2023). doi:10.48550/arXiv.2307.09288
- [138] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 16022–16076. doi:10.18653/v1/2024.acl-long.850
- [139] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems 30*. 5998–6008. doi:10.48550/arXiv.1706.03762

- [140] Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc Le, and Thang Luong. 2024. FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 13697–13720. doi:10.18653/v1/2024.findings-acl.813
- [141] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *CoRR* abs/2305.16291 (2023). doi:10.48550/arXiv.2305.16291
- [142] Lei Wang, Chengbang Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science* 18, 6, Article 186345 (2024). doi:10.1007/s11704-024-40231-1
- [143] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*. 2609–2634. doi:10.18653/v1/2023.acl-long.147
- [144] Lu Wang, Fangkai Yang, Chaoyun Qian, Junting Lu, Jiaxu Qian, Shilin He, Pu Zhao, Bo Qiao, He Huang, Si Qin, Qisheng Su, Jiayi Ye, Yudi Zhang, Jian-Guang Lou, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2025. Large Action Models: From Inception to Implementation. *Transactions on Machine Learning Research* (2025). doi:10.48550/arXiv.2412.10047
- [145] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 50208–50232. doi:10.5555/3692070.3694124
- [146] Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. 2024. SciBench: Evaluating College-Level Scientific Problem-Solving Abilities of Large Language Models. *CoRR* abs/2307.10635 (2024). doi:10.48550/arXiv.2307.10635
- [147] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *CoRR* abs/2407.16741 (2024). doi:10.48550/ARXIV.2407.16741
- [148] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*. doi:10.48550/arXiv.2203.11171
- [149] Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. 2024. OfficeBench: Benchmarking Language Agents across Multiple Applications for Office Automation. *arXiv preprint arXiv:2407.19056* (2024). doi:10.48550/arXiv.2407.19056
- [150] Zhiruo Wang, Daniel Fried, and Graham Neubig. 2024. TroVE: Inducing Verifiable and Efficient Toolboxes for Solving Programmatic Tasks. In *Proceedings of the 41st International Conference on Machine Learning*. doi:10.48550/ARXIV.2401.12869
- [151] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. 2025. RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning. arXiv:2504.20073 doi:10.48550/arXiv.2504.20073
- [152] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent Abilities of Large Language Models. *CoRR* abs/2206.07682 (2022). doi:10.48550/arXiv.2206.07682
- [153] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 35. 24824–24837. doi:10.48550/arXiv.2201.11903
- [154] Jason Weston and Sainbayar Sukhbaatar. 2023. System 2 Attention (is something you might need too). *CoRR* abs/2311.11829 (2023). doi:10.48550/ARXIV.2311.11829
- [155] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, Jiale Zhang, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. *CoRR* abs/2308.08155 (2023). doi:10.48550/ARXIV.2308.08155
- [156] Yue Wu, Yewen Fan, So Yeon Min, Shrimai Prabhumoye, Stephen McAleer, Yonatan Bisk, Ruslan Salakhutdinov, Yuanzhi Li, and Tom Mitchell. 2024. AgentKit: Structured LLM Reasoning with Dynamic Graphs. *CoRR* abs/2404.11483 (2024). arXiv:2404.11483 [cs.AI] doi:10.48550/arXiv.2404.11483
- [157] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. 2024. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. *CoRR* abs/2402.07456 (2024). arXiv:2402.07456 doi:10.48550/arXiv.2402.07456
- [158] Zhiheng Xi, Chenyang Liao, Guanyu Li, Yajie Yang, Wenxiang Chen, Zhihao Zhang, Binghai Wang, Senjie Jin, Yuhao Zhou, Jian Guan, Wei Wu, Tao Ji, Tao Gui, Qi Zhang, and Xuanjing Huang. 2025. AgentPRM: Process Reward Models for LLM Agents via Step-Wise Promise and Progress. arXiv:2511.08325 doi:10.48550/arXiv.2511.08325
- [159] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE, Article FSE037 (2025), 801–824 pages. doi:10.1145/3715754
- [160] Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. 2025. Agent0: Unleashing Self-Evolving Agents from Zero Data via Tool-Integrated Reasoning. arXiv:2511.16043 doi:10.48550/arXiv.2511.16043

- [161] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. 2024. TravelPlanner: A Benchmark for Real-World Planning with Language Agents. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 54590–54613. <https://proceedings.mlr.press/v235/xie24j.html>
- [162] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. 2022. An Explanation of In-context Learning as Implicit Bayesian Inference. In *Proceedings of the Tenth International Conference on Learning Representations*. doi:10.48550/arXiv.2111.02080
- [163] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. *arXiv preprint arXiv:2404.07972* (2024). doi:10.48550/arXiv.2404.07972
- [164] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. 2023. ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models. *CoRR abs/2305.18323* (2023). doi:10.48550/arXiv.2305.18323
- [165] Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Yiqing Xie, Shuyan Zhou, and Graham Neubig. 2024. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. *CoRR abs/2412.14161* (2024). doi:10.48550/arXiv.2412.14161
- [166] Yiheng Xu, Hongjin Su, Chen Xing, Boyu Mi, Qian Liu, Weijia Shi, Binyuan Hui, Fan Zhou, Yitao Liu, Tianbao Xie, Zhoujun Cheng, Siheng Zhao, Lingpeng Kong, Bailin Wang, Caiming Xiong, and Tao Yu. 2024. Lemur: Harmonizing Natural Language and Code for Language Agents. *CoRR abs/2310.06830* (2024). doi:10.48550/arXiv.2310.06830
- [167] Zhenghai Xue, Longtao Zheng, Qian Liu, Yingru Li, Xiaosen Zheng, Zejun Ma, and Bo An. 2025. SimpleTIR: End-to-End Reinforcement Learning for Multi-Turn Tool-Integrated Reasoning. *CoRR abs/2509.02479* (2025). doi:10.48550/arXiv.2509.02479
- [168] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback. *arXiv preprint arXiv:2306.14898* (2023). doi:10.48550/arXiv.2306.14898
- [169] Jianwei Yang, Reuben Tan, Qianhui Wu, Ruijie Zheng, Baolin Peng, Yongyuan Liang, Yu Gu, Mu Cai, Seonghyeon Ye, Joel Jang, Yuquan Deng, Lars Liden, and Jianfeng Gao. 2025. Magma: A Foundation Model for Multimodal AI Agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. doi:10.1109/CVPR52734.2025.01325
- [170] Rui Yang, Lin Song, Yanwei Li, Sijie Zhao, Yixiao Ge, Xiu Li, and Ying Shan. 2023. GPT4Tools: Teaching Large Language Model to Use Tools via Self-instruction. *CoRR abs/2305.18752* (2023). doi:10.48550/arXiv.2305.18752
- [171] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 2369–2380. doi:10.18653/v1/D18-1259
- [172] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R. Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *CoRR abs/2406.12045*. doi:10.48550/arXiv.2406.12045
- [173] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. 11809–11822. doi:10.48550/arXiv.2305.10601
- [174] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the International Conference on Learning Representations*. doi:10.48550/arXiv.2210.03629
- [175] Weiran Yao, Shelby Heinecke, Juan Carlos Niebles, Zhiwei Liu, Yihao Feng, Le Xue, Rithesh Murthy, Zeyuan Chen, Jianguo Zhang, Devansh Arpit, Ran Xu, Phil Mui, Huan Wang, Caiming Xiong, and Silvio Savarese. 2024. Retroformer: Retrospective Large Language Agents with Policy Gradient Optimization. In *Proceedings of the International Conference on Learning Representations*. doi:10.48550/arXiv.2308.02151
- [176] Junjie Ye, Guanyu Li, SongYang Gao, Caishuang Huang, Yilong Wu, Sixian Li, Xiaoran Fan, Shihan Dou, Tao Ji, Qi Zhang, Tao Gui, and Xuanjing Huang. 2025. ToolEyes: Fine-Grained Evaluation for Tool Learning Capabilities of Large Language Models in Real-world Scenarios. In *Proceedings of the 31st International Conference on Computational Linguistics*. Association for Computational Linguistics, Abu Dhabi, UAE, 156–187. <https://aclanthology.org/2025.coling-main.12/>
- [177] Junjie Ye, Sixian Li, Guanyu Li, Caishuang Huang, Songyang Gao, Yilong Wu, Qi Zhang, Tao Gui, and Xuanjing Huang. 2024. ToolSword: Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 2181–2211. doi:10.18653/v1/2024.acl-long.119
- [178] Junjie Ye, Yilong Wu, Songyang Gao, Caishuang Huang, Sixian Li, Guanyu Li, Xiaoran Fan, Qi Zhang, Tao Gui, and Xuanjing Huang. 2024. RoTBench: A Multi-Level Benchmark for Evaluating the Robustness of Large Language Models in Tool Learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 313–333. doi:10.18653/v1/2024.emnlp-main.19
- [179] Jiazhen Yuan, Wei Zhao, and Zhengbiao Bai. 2025. Graph-Enhanced Policy Optimization in LLM Agent Training. *CoRR abs/2510.26270* (2025). arXiv:2510.26270 doi:10.48550/arXiv.2510.26270
- [180] Siyu Yuan, Zehui Chen, Zhiheng Xi, Junjie Ye, Zhengyin Du, and Jiecao Chen. 2025. Agent-R: Training Language Model Agents to Reflect via Iterative Self-Training. doi:10.48550/arXiv.2501.11425
- [181] Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Yongliang Shen, Kan Ren, Dongsheng Li, and Deqing Yang. 2025. EASYTOOL: Enhancing LLM-Based Agents with Concise Tool Instruction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association*

- for *Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 951–972. doi:10.18653/v1/2025.naacl-long.44
- [182] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Miami, Florida, USA, 1467–1490. doi:10.18653/v1/2024.findings-emnlp.79
- [183] Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. 2024. AgentTuning: Enabling Generalized Agent Abilities for LLMs. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 3053–3077. doi:10.18653/v1/2024.findings-acl.181
- [184] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 10471–10506. doi:10.18653/v1/2024.findings-acl.624
- [185] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2025. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 70, 20 pages. doi:10.1145/3706598.3713600
- [186] Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Jat Long long, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, Rui Lu, Hongning Wang, Jie Tang, and Yuxiao Dong. 2025. AgentRL: Scaling Agentic Reinforcement Learning with a Multi-Turn, Multi-Task Framework. *arXiv preprint arXiv:2510.04206* (2025). doi:10.48550/arXiv.2510.04206
- [187] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Zhiwei Liu, Yihao Feng, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. 2025. xLAM: A Family of Large Action Models to Empower AI Agent Systems. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Albuquerque, New Mexico, 11583–11597. doi:10.18653/v1/2025.naacl-long.578
- [188] Qi Zhang, Shouqing Yang, Lirong Gao, Hao Chen, Xiaomeng Hu, Jinglei Chen, Jiexiang Wang, Sheng Guo, Bo Zheng, Haobo Wang, and Junbo Zhao. 2025. LeTS: Learning to Think-and-Search via Process-and-Outcome Reward Hybridization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*. Association for Computational Linguistics, Suzhou, China, 5109–5122. doi:10.18653/v1/2025.emnlp-main.257
- [189] Tianjun Zhang, Shishir G. Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E. Gonzalez. 2024. RAFT: Adapting Language Model to Domain Specific RAG. In *Proceedings of COLM 2024*. doi:10.48550/arXiv.2403.10131
- [190] Zeyu Zhang, Quanyu Dai, Xiaohu Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A Survey on the Memory Mechanism of Large Language Model-based Agents. *ACM Transactions on Information Systems* 43, 6, Article 155 (2025), 47 pages. doi:10.1145/3748302
- [191] Zhixin Zhang, Leqi Lei, Lindong Wu, Rui Sun, Yongkang Huang, Chong Long, Xiao Liu, Xuanyu Lei, Jie Tang, and Minlie Huang. 2024. SafetyBench: Evaluating the Safety of Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 15537–15553. doi:10.18653/v1/2024.acl-long.830
- [192] Zaibin Zhang, Yongting Zhang, Lijun Li, Hongzhi Gao, Lijun Wang, Huchuan Lu, Feng Zhao, Yu Qiao, and Jing Shao. 2024. PsySafe: A Comprehensive Framework for Psychological-based Attack, Defense, and Evaluation of Multi-agent System Safety. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 15202–15231. doi:10.18653/v1/2024.acl-long.812
- [193] Peng Zhao, Zijian Jin, and Ning Cheng. 2023. An In-depth Survey of Large Language Model-based Artificial Intelligence Agents. *CoRR* abs/2309.14365 (2023). doi:10.48550/arXiv.2309.14365
- [194] Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2023. Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models. In *The Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2310.06117
- [195] Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. 2024. Synapse: Trajectory-as-Exemplar Prompting with Memory for Computer Control. In *Proceedings of the Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2306.07863
- [196] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. 2025. DeepResearcher: Scaling Deep Research via Reinforcement Learning in Real-world Environments. *arXiv preprint arXiv:2504.03160* (2025). doi:10.48550/arXiv.2504.03160
- [197] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2023. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 62138–62160. doi:10.48550/arXiv.2310.04406
- [198] Caishen Zhou, Yihong Tang, Kehai Chen, Xuefeng Bai, Shuhan Qi, Li Shen, and Min Zhang. 2026. From Fragmentation to Systematic Design: Architecting LLM-Based Multi-Agent Systems. *TechRxiv* (2026). doi:10.36227/techrxiv.176827304.41872996/v2
- [199] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. 2023. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. In *The Eleventh International Conference on Learning Representations*. doi:10.48550/arXiv.2205.10625
- [200] Shuyang Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of the Twelfth International*

- Conference on Learning Representations*. doi:10.48550/arXiv.2307.13854
- [201] Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. ToolQA: A Dataset for LLM Question Answering with External Tools. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*. Curran Associates, Inc., 50117–50143. doi:10.48550/arXiv.2306.13304
- [202] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, David Lo, Binyuan Hui, Niklas Muennighoff, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro von Werra. 2024. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *arXiv preprint arXiv:2406.15877* (2024). doi:10.48550/arXiv.2406.15877

A Summary of Related Works

Table 5 summarizes the major methodological papers discussed in this survey.

Table 5. Summary of representative methodological papers on agentic tool use.

Paper	Year	Training Signal	Tool Scope	Multi-turn	Multi-tool	Setting	Affiliation
ReAct [174]	2023	Prompting / ICL	External actions	✓	✓	Reason-act loop	Princeton
Reflexion [125]	2023	Prompting + verbal feedback	External actions	✓	✓	Self-reflective interaction	Princeton
CRITIC [40]	2024	Prompting + interactive critique	Verification tools	✓	✓	Verification and correction	Tsinghua
LATS [197]	2023	Prompting + search	Search	✓	✓	Trajectory tree search	UIUC
ReWOO [164]	2023	Prompting / planning	General external tools	✓	✓	Decoupled planning	MSU
HuggingGPT [122]	2023	Prompting / orchestration	Expert models	✓	✓	LLM tool orchestration	ZJU
LLM-Compiler [64]	2024	Prompting / planning	Function calls	✓	✓	Parallel decomposition	UC Berkeley
PAL [37]	2023	Prompting / code generation	Code execution	✗	✗	Program-aided reasoning	CMU
PoT [23]	2023	Prompting / code generation	Code execution	✗	✗	Program-of-thought	Waterloo
ViperGPT [131]	2023	Prompting / code generation	Vision APIs	✗	✓	Visual reasoning	Columbia
Code as Policies [75]	2022	Prompting / code generation	Control APIs	✗	✓	Embodied control	Google
LATM [14]	2024	Prompting / tool creation	Python tools	✓	✓	Tool creation and reuse	Stanford
Toolformer [121]	2023	Self-supervised API annotation	APIs	✗	✓	API self-annotation	Meta AI
AutoAct [110]	2024	Synthetic planning trajectories	Tool-augmented agents	✓	✓	Trajectory synthesis	ZJU
ToolACE [79]	2025	Self-evolving synthetic data	Function calls	✓	✓	API/dialogue synthesis	SJTU
APIGen [81]	2024	Verifiable synthetic data	Function calls	✗	✓	API-call generation	Salesforce AI Research
ToolLLM [112]	2023	Instruction tuning	RESTful APIs	✓	✓	Large-scale API use	Tsinghua
API-Bank [73]	2023	Benchmark-driven supervision	APIs	✓	✓	Planning and API use	Alibaba DAMO
GPT4Tools [170]	2023	Synthetic data + instruction	Tool collections	✗	✓	Tool-use tuning	Tsinghua SIGS
Functionary [88]	2024	Instruction tuning	Function calls	✓	✓	Function calling	MeetKai
FireAct [16]	2023	SFT on reasoning trajectories	General external tools	✓	✓	Reason-act tuning	System2
Agent-FLAN [26]	2024	SFT with negative samples	Agent actions	✓	✓	Agent tuning	USTC
ToolAlign [27]	2024	Alignment-oriented tuning	Tool-use environments	✓	✓	HHA alignment	RUC
ToRA [41]	2024	Rationale-augmented training	Mathematical tools	✓	✓	Tool-augmented math	Tsinghua
MetaTool [51]	2024	Supervised tuning	Tool selection	✗	✓	Tool selection	XJTU
ReTool [35]	2025	Reinforcement learning	General external tools	✗	✓	Strategic tool use	ByteDance Seed
ToolExpander [19]	2025	RL + hard-sampling curriculum	General external tools	✓	✓	Capability expansion	OPPO
ToolRL [107]	2025	Outcome-reward RL	General external tools	✗	✓	Tool-use optimization	UIUC
Agent0 [160]	2025	RL + self-generated curriculum	General external tools	✓	✓	Self-evolving tool use	UNC
Search-R1 [59]	2025	Reinforcement learning	Search / retrieval tools	✓	✗	Search-oriented reasoning	UIUC
DeepResearcher [196]	2025	Reinforcement learning	Web research tools	✓	✓	Deep web research	SJTU
AgentPRM [158]	2025	Process reward modeling	General external tools	✓	✓	Stepwise agent evaluation	FDU
SimpleTIR [167]	2025	End-to-end RL	General external tools	✓	✓	Joint reason-tool execution	NTU
VerTool [56]	2025	Agentic RL	Tool selector + memory	✓	✓	Holistic optimization	UWaterloo
DeepAgent [74]	2025	Agentic RL framework	Scalable toolsets	✓	✓	General tool agent	RUC
ToolRM [2]	2025	Reward modeling	Tool-use trajectories	✓	✓	Tool-call reward modeling	IBM Research
Agent Q [106]	2024	Guided MCTS + self-critique	Web environment	✓	✓	Interactive web agents	MultiOn
DigiRL [6]	2024	Offline-to-online RL	GUI / device control	✓	✓	Android interaction	UC Berkeley